

DOCUMENTACAO_COMPLETA_PTHREADS

Programação Paralela em C com Pthreads: Documentação Técnica Completa

Autor: Especialista em Programação Paralela

Data: 2026

Escopo: Guia técnico, didático e prático de Programação Paralela em C

Índice

1. [Fundamentos](#)
 2. [Processos](#)
 3. [Threads e pthread_create](#)
 4. [pthread_join](#)
 5. [pthread_exit](#)
 6. [Seção Crítica e Mutex](#)
 7. [Semáforos](#)
 8. [Variáveis de Condição](#)
 9. [Barreiras](#)
 10. [Condição de Corrida](#)
 11. [Deadlock](#)
 12. [Starvation \(Inanição\)](#)
 13. [Violação de Atomicidade](#)
 14. [Escalonamento](#)
 15. [Paralelismo de Dados](#)
 16. [Paralelismo de Tarefas](#)
 17. [Problema: Produtor-Consumidor](#)
 18. [Problema: Leitores-Escritores](#)
 19. [Problema: Jantar dos Filósofos](#)
 20. [Balanceamento de Carga](#)
 21. [Saco de Tarefas \(Bag of Tasks\)](#)
-

Fundamentos

Execução Sequencial vs. Concorrente vs. Paralela

Definição

- **Execução Sequencial:** Tarefas executadas uma por vez, em ordem estrita.
- **Execução Concorrente:** Múltiplas tarefas em progresso (time-sharing em 1 CPU).
- **Execução Paralela:** Tarefas simultâneas em múltiplos processadores.

Comparação

Modelo	Hardware	Comportamento
Sequencial	1 CPU	$T1 \rightarrow T2 \rightarrow T3$
Concorrente	1 CPU	Intercalação de tarefas
Paralelo	Múltiplos CPUs	Execução simultânea real

Ganho de Desempenho

O paralelismo é vantajoso apenas se o ganho superar o overhead. Tarefas muito pequenas podem ser mais lentas paralelizadas.

Processos

Definição

Um processo é um programa em execução com seu próprio espaço de endereçamento isolado.

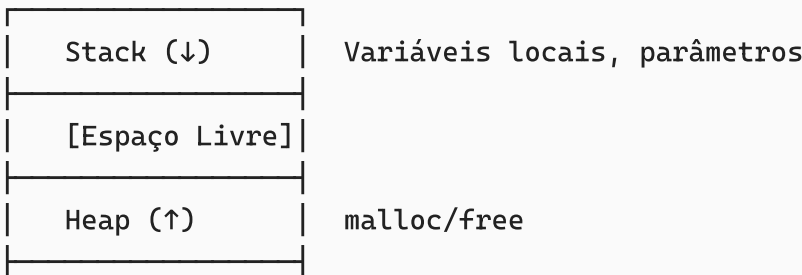
Objetivo

Permitir múltiplos programas rodarem independentemente com isolamento total de memória.

Funcionamento

O SO gerencia processos através do **Bloco de Controle de Processo (PCB)**.

Layout de Memória (Formato ELF)



.bss / .data	Dados estáticos
.rodata	Constantes
.text	Instruções

Estados de um Processo

1. **Novo:** Criado, aguardando fila
2. **Pronto:** Aguardando CPU
3. **Execução:** Em processamento
4. **Bloqueado:** Aguardando E/S
5. **Encerrado:** Finalizado

Sintaxe: fork()

```
#include <sys/types.h>
#include <unistd.h>

pid_t fork(void);
```

Parâmetros e Retorno

Retorno	Significado
0	Processo filho
>0	PID do filho (pai recebe)
-1	Erro

Exemplo Completo

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork falhou");
        exit(1);
    } else if (pid == 0) {
        /* CÓDIGO DO FILHO */
    }
```

```
    printf("Sou o filho (PID: %d)\n", getpid());
    exit(0);
} else {
    /* CÓDIGO DO PAI */
    printf("Sou o pai (PID: %d), aguardando filho...\n", getpid());
    wait(NULL); /* Sincronização */
    printf("Filho terminou.\n");
}
return 0;
}
```

Explicação Linha por Linha

1. `fork()` divide o fluxo em dois processos
2. Filho recebe retorno 0, entra no `else if`
3. Pai recebe PID do filho (>0), entra no `else`
4. `wait()` bloqueia pai até filho terminar
5. Cada processo tem seu próprio `getpid()`

Saída Esperada

```
Sou o pai (PID: 1234), aguardando filho...
Sou o filho (PID: 1235)
Filho terminou.
```

Problemas Comuns

- **Processo Zumbi:** Filho termina, pai não chama `wait()` → filho fica na tabela de processos
- **Esquecer `wait()`:** Pai continua, deixando filho órfão
- **Confundir `fork()` com `return`:** `fork` retorna EM AMBOS os processos

Erros Frequentes em Provas

- Usar `if (pid == 0)` para identificar pai (incorreto, filho recebe 0)
- Não chamar `wait()` para sincronizar
- Tentar acessar variáveis do pai no filho após `fork` sem entender isolamento de memória

Substituição de Imagem: A Família `exec()`

Definição

A família de funções `exec()` substitui o código, dados e pilha do processo atual por um novo programa executável, mantendo o mesmo PID.

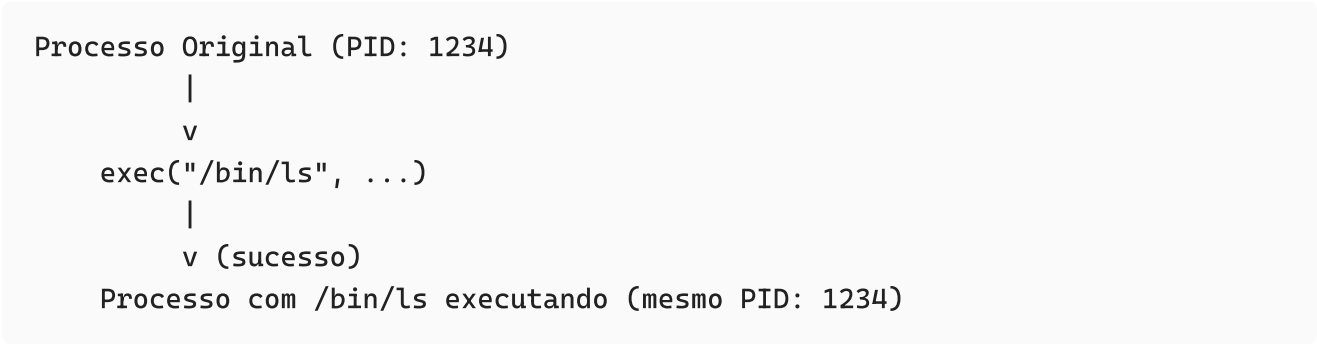
Objetivo

Permitir que um processo filho execute um programa completamente diferente do programa original (comum após `fork()`).

Funcionamento

- **Sem retorno:** Se bem-sucedido, `exec()` nunca retorna (o novo programa sobrescreve o anterior)
- **Preservação:** PID, file descriptors e sinais são preservados
- **Memória:** Espaço de endereçamento é totalmente substituído
- **Erro:** Se falhar, retorna -1 e continua no programa original

Diagrama de Fluxo



Variantes Principais

Variante	Formato	Localização
execl	<code>execl(path, arg0, arg1, ..., NULL)</code>	Caminho absoluto
execv	<code>execv(path, argv[])</code>	Vetor de argumentos
execlp	<code>execlp(file, arg0, ..., NULL)</code>	Busca em PATH
execvp	<code>execvp(file, argv[])</code>	Vetor + busca PATH
execle	<code>execle(path, arg0, ..., NULL, envp[])</code>	Com environment
execve	<code>execve(path, argv[], envp[])</code>	Vetor + environment

Sintaxe

```
#include <unistd.h>

int execl(const char *path, const char *arg0, ..., NULL);

int execv(const char *path, char *const argv[]);
```

```
int execlp(const char *file, const char *arg0, ..., NULL);

int execvp(const char *file, char *const argv[]);

int execl(const char *path, const char *arg0, ..., NULL, char *const envp[]);

int execve(const char *path, char *const argv[], char *const envp[]);
```

Parâmetros

Parâmetro	Significado
path	Caminho absoluto do executável (ex: /bin/ls)
file	Nome do executável (busca em PATH)
arg0, arg1, ...	Argumentos da linha de comando (arg0 = nome do programa)
argv[]	Vetor de argumentos (última entrada = NULL)
envp[]	Vetor de variáveis de ambiente (última = NULL)

Valor de Retorno

- **Nunca retorna em sucesso** (processo é substituído)
- **-1**: Erro (com errno definido)

Exemplo Completo 1: exec() Simples

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    printf("Antes de exec()\n");

    /* execl com argumentos */
    execl("/bin/echo", "echo", "Olá do exec!", NULL);

    /* Nunca atinge aqui se exec() suceder */
    printf("Após exec() - nunca é executado\n");
    perror("exec falhou");

    return 1;
}
```

Exemplo Completo 2: fork() + exec()

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork falhou");
        exit(1);
    } else if (pid == 0) {
        /* FILHO: Substituir processo por /bin/ls */
        char *argv[] = {"ls", "-la", "/tmp", NULL};

        printf("Filho: Executando 'ls -la /tmp'\n");

        /* Substitui este processo */
        execv("/bin/ls", argv);

        /* Nunca atinge aqui se exec() suceder */
        perror("execv falhou");
        exit(1);
    } else {
        /* PAI: Aguarda filho */
        printf("Pai: Aguardando filho (PID: %d)\n", pid);
        wait(NULL);
        printf("Pai: Filho terminou.\n");
    }

    return 0;
}

```

Exemplo Completo 3: execvp com PATH

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        /* FILHO: Busca 'python3' em PATH */
        char *argv[] = {"python3", "-c", "print('Olá Python!')", NULL};

        /* execvp busca em PATH */

```

```

    execvp("python3", argv);

    /* Se chegou aqui, erro */
    perror("execvp falhou");
    exit(1);
} else {
    wait(NULL);
    printf("Programa Python terminou.\n");
}

return 0;
}

```

Exemplo Completo 4: Capturar Saída com Pipes

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipe_fd[2];

    if (pipe(pipe_fd) < 0) {
        perror("pipe falhou");
        exit(1);
    }

    pid_t pid = fork();

    if (pid == 0) {
        /* FILHO: Redireciona stdout para pipe */
        close(pipe_fd[0]); /* Fecha leitura */
        dup2(pipe_fd[1], STDOUT_FILENO); /* Redireciona stdout */
        close(pipe_fd[1]);

        /* Executa 'date' */
        execl("/bin/date", "date", NULL);

        perror("execl falhou");
        exit(1);
    } else {
        /* PAI: Lê do pipe */
        close(pipe_fd[1]); /* Fecha escrita */

        char buffer[256];
        ssize_t bytes = read(pipe_fd[0], buffer, sizeof(buffer) - 1);

        if (bytes > 0) {

```

```

        buffer[bytes] = '\0';
        printf("Data do sistema: %s", buffer);
    }

    close(pipe_fd[0]);
    wait(NULL);
}

return 0;
}

```

Explicação do Código (Exemplo 2)

1. **Linha `fork()`:** Cria processo filho
2. **Linha `if (pid == 0)`:** Apenas filho entra neste bloco
3. **Linha `argv[]`:** Define argumentos (`argv[0]` = nome do programa)
4. **Linha `execv()`:** Substitui filho por `/bin/ls`
5. **Linhas após `execv`:** Nunca executadas se `exec` suceder
6. **Linha `wait()`:** Pai aguarda filho terminar

Saída Esperada (Exemplo 2)

```

Pai: Aguardando filho (PID: 1235)
Filho: Executando 'ls -la /tmp'
[Saída de 'ls -la /tmp']
Pai: Filho terminou.

```

Comparação: `exec()` vs. `Não-exec()`

```

/* SEM exec(): novo processo executa mesmo código */
if (fork() == 0) {
    for (int i = 0; i < 5; i++) {
        printf("Trabalhando... %d\n", i);
        sleep(1);
    }
    exit(0);
}

/* COM exec(): novo processo executa programa diferente */
if (fork() == 0) {
    execl("/bin/sleep", "sleep", "5", NULL);
    exit(1); /* Erro se exec falhar */
}

```

Problemas Comuns

- **Esquecer NULL ao final:** `execl("/bin/ls", "ls", NULL)` é obrigatório
- **Caminho incorreto:** `/bin/ls` é correto; `ls` não funciona com `execl`
- **argv[0] deve ser nome do programa:** Convenção importante
- **Não fechar file descriptors:** Deixar descritores abertos pode travar processos
- **Não verificar retorno:** `exec()` deveria estar seguido de `exit()` ou `perror()`

Boas Práticas

1. **Sempre usar `fork()` antes de `exec()`** para não perder o processo original
2. **Verificar retorno de `fork()`** antes de chamar `exec()`
3. **Sempre terminar lista de argumentos com NULL**
4. **Usar `execvp()` quando possível** (busca em PATH automaticamente)
5. **Redirecionar file descriptors ANTES de `exec()`** para capturar saída
6. **Usar `wait()` para sincronizar** e coletar status

Diferenças: `execl` vs. `execv`

```
/* execl: lista de argumentos manual */
execl("/bin/ls", "ls", "-la", "/tmp", NULL);

/* execv: vetor de argumentos */
char *argv[] = {"ls", "-la", "/tmp", NULL};
execv("/bin/ls", argv);
/* Resultado idêntico */
```

Diference: Busca em PATH

```
/* execlp: busca 'python3' em PATH */
execlp("python3", "python3", "script.py", NULL);

/* execl: exige caminho absoluto */
execl("/usr/bin/python3", "python3", "script.py", NULL);
```

Erros Frequentes em Provas

- Chamar `exec()` na thread principal sem `fork()`
 - Esquecer que `exec()` **nunca retorna em sucesso**
 - Tentar usar variáveis após `exec()` (são perdidas)
 - Confundir `arg0` com `argv[0]`
 - Não usar NULL para terminar lista de argumentos
 - Executar programa que não existe (erro silencioso)
-

Threads

Definição

Threads (LWP - Light Weight Process) são fluxos de execução independentes que compartilham o **mesmo processo** e espaço de endereçamento.

Objetivo

Permitir paralelismo com baixo custo de criação e facilitar comunicação (compartilham memória).

Funcionamento

Compartilhamento vs. Contexto Privado

Compartilhado	Privado por Thread
Variáveis globais	PC (Program Counter)
Heap	Registradores
Arquivos abertos	Stack
Sinais	TID (Thread ID)

Risco Principal

Acesso simultâneo a dados compartilhados sem sincronização causa **race conditions**.

pthread_create

Definição

Função que cria uma nova thread dentro do processo atual.

Objetivo

Disparar múltiplos fluxos de execução para paralelismo real.

Funcionamento

- **Criação:** Assíncrona (thread chamadora continua imediatamente)
- **Stack:** Cada thread tem sua própria pilha
- **Espaço de Endereçamento:** Compartilhado com todas as threads
- **Escalonamento:** SO decide quando executar cada thread

Sintaxe

```
int pthread_create(  
    pthread_t *thread,  
    const pthread_attr_t *attr,  
    void *(*start_routine)(void *),  
    void *arg  
);
```

Parâmetros

Parâmetro	Significado
*thread	Ponteiro onde armazenar ID da thread
*attr	Atributos (NULL para padrão)
start_routine	Função que a thread executará
arg	Um único argumento (void *)

Valor de Retorno

- **0:** Sucesso
- **Erro:** Código de erro numérico (EAGAIN, ENOMEM, etc.)

Compilação

```
gcc programa.c -o programa -lpthread
```

Exemplo Completo

```
#include <pthread.h>  
#include <stdio.h>  
#include <stdlib.h>  
  
typedef struct {  
    int id;  
    int valor;  
} ThreadData;  
  
void *tarefa(void *arg) {  
    ThreadData *data = (ThreadData *)arg;  
    printf("Thread %d com valor %d\n", data->id, data->valor);  
    free(data);  
    return NULL;  
}
```

```

int main() {
    pthread_t threads[2];

    for (int i = 0; i < 2; i++) {
        ThreadData *td = malloc(sizeof(ThreadData));
        td->id = i;
        td->valor = i * 100;

        if (pthread_create(&threads[i], NULL, tarefa, td) != 0) {
            perror("Erro ao criar thread");
            return 1;
        }
    }

    /* Aguardar todas as threads terminarem */
    for (int i = 0; i < 2; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Todas as threads completadas.\n");
    return 0;
}

```

Explicação do Código

1. **Linhas ThreadData:** Struct para passar múltiplos dados
2. **Linha na tarefa():** Recebe `void *`, faz cast para acessar membros
3. **Linha malloc:** Aloca memória NO HEAP (não pode usar variável local)
4. **Linha pthread_create:** Dispara thread com argumento alocado
5. **Linha free:** Thread libera sua própria memória antes de terminar
6. **Linha pthread_join:** Main aguarda todas as threads

Saída Esperada

```

Thread 0 com valor 0
Thread 1 com valor 100
Todas as threads completadas.

```

Problemas Comuns

- **Passar endereço de variável local do loop:** Todas as threads veem o valor final de `i`
- **Esquecer -lpthread:** Erro de ligação
- **Não alocar memória no heap:** Variável local é destruída antes da thread ler

Boas Práticas

1. Sempre verificar retorno de `pthread_create`
2. Para múltiplos argumentos, usar `struct`
3. Alocar no heap se dados forem usados após `thread_create` retornar
4. Usar `pthread_join` em TODAS as threads

Erros Frequentes em Provas

- Passar `&i` em um loop (race condition)
 - Esquecer de compilar com `-lpthread`
 - Não entender que a criação é assíncrona (main pode terminar antes das threads)
-

pthread_join

Definição

Função que bloqueia a thread chamadora até que a thread alvo termine.

Objetivo

Sincronizar threads e coletar resultados.

Funcionamento

- **Bloqueio:** Thread que chama `join` suspende
- **Coleta:** Recebe valor de retorno da thread alvo
- **Limpeza:** Libera recursos da thread encerrada

Sintaxe

```
int pthread_join(pthread_t thread, void **retval);
```

Parâmetros

Parâmetro	Significado	
thread	ID da thread a aguardar	
**retval	Ponteiro para receber valor de retorno	

Valor de Retorno

- **0:** Sucesso
- **Erro:** EINVAL, ESRCH, etc.

Exemplo Completo

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

void *calcular_quadrado(void *arg) {
    int n = *(int *)arg;
    int *resultado = malloc(sizeof(int));
    *resultado = n * n;
    free(arg); /* Libera argumento recebido */
    return (void *)resultado;
}

int main() {
    pthread_t tid;
    int numero = 5;
    void *ret = NULL;

    /* Alocar para passar para thread */
    int *param = malloc(sizeof(int));
    *param = numero;

    pthread_create(&tid, NULL, calcular_quadrado, param);

    /* pthread_join recebe endereço de void * */
    pthread_join(tid, &ret);

    printf("Resultado: %d\n", *(int *)ret);
    free(ret); /* Obrigatório liberar memória retornada */

    return 0;
}
```

Explicação do Código

1. **Linha calcular_quadrado:** Aloca memória para resultado no heap
2. **Linha return:** Retorna endereço da memória alocada
3. **Linha pthread_join:** Recebe &ret (endereço do ponteiro void *)
4. **Linha depois do join:** ret agora aponta para a memória do resultado
5. **Linha printf:** Faz cast para int* e desreferencia

Saída Esperada

Resultado: 25

Problemas Comuns

- **Dangling Pointer:** Retornar endereço de variável local da thread
- **Memory Leak:** Não liberar memória retornada
- **Zombie Thread:** Não chamar join deixa recursos presos

Boas Práticas

1. **SEMPRE** chamar `pthread_join` ou `pthread_detach`
 2. Se retornar dados, alocar no **HEAP**
 3. Responsabilidade do join: liberar memória retornada
-

pthread_exit

Definição

Termina uma thread de forma limpa.

Sintaxe

```
void pthread_exit(void *retval);
```

Objetivo

Permitir que a thread retorne um valor sem encerrar o processo.

Comparação

Função	Efeito
<code>return</code>	Termina função (thread também)
<code>pthread_exit()</code>	Termina apenas a thread
<code>exit()</code>	Termina TODO o processo

Exemplo

```
void *tarefa(void *arg) {  
    int *resultado = malloc(sizeof(int));  
    *resultado = 42;  
    pthread_exit((void *)resultado); /* Termina thread, retorna valor */  
}
```

Nota Especial: pthread_exit() na main

Se `main` chamar `pthread_exit`:

- Main termina
 - Processo permanece vivo
 - Threads filhas continuam executando
 - Processo encerra quando última thread terminar
-

Passagem de Argumentos para Threads

Definição

Passagem de argumentos refere-se aos mecanismos de enviar dados de uma thread (geralmente a main) para outras threads executadas via `pthread_create`.

Objetivo

Permitir que threads recebam informações necessárias para sua execução (ID, configurações, dados para processar).

Desafio Principal

`pthread_create` aceita apenas **um argumento único** do tipo `void *`. Portanto, técnicas são necessárias para passar:

- **Um valor simples** (int, char, etc.)
- **Múltiplos valores** (usar struct ou malloc)

Problema: Race Condition ao Passar Endereço de Loop

Exemplo Problemático

```
#include <pthread.h>
#include <stdio.h>

void *tarefa(void *arg) {
    int valor = *(int *)arg;
    printf("Thread recebeu: %d\n", valor);
    return NULL;
}

int main() {
    pthread_t threads[5];
```

```

/* PROBLEMA: Passar &i diretamente */
for (int i = 0; i < 5; i++) {
    pthread_create(&threads[i], NULL, tarefa, &i); /* &i muda! */
}

for (int i = 0; i < 5; i++) {
    pthread_join(threads[i], NULL);
}

return 0;
}

```

Saída (Imprevisível):

```

Thread recebeu: 5
Thread recebeu: 5
Thread recebeu: 5
Thread recebeu: 5
Thread recebeu: 5

```

Por que? Todas as threads veem o valor FINAL de `i` (5), pois o endereço `&i` é o mesmo.

Técnica 1: Passar Valor Simples (Casting)

Quando Usar

Quando passar um único valor pequeno (int, char, pequeno index).

Funcionamento

Fazer cast do valor para `void *` e vice-versa. Funciona para valores que cabem em um ponteiro.

Exemplo: Passagem de Inteiro

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

void *tarefa(void *arg) {
    /* Converter void * para int */
    int id = (int)(intptr_t)arg;
    printf("Thread ID: %d\n", id);
    return NULL;
}

```

```
int main() {
    pthread_t threads[5];

    for (int i = 0; i < 5; i++) {
        /* Converter int para void * */
        pthread_create(&threads[i], NULL, tarefa, (void *)(intptr_t)i);
    }

    for (int i = 0; i < 5; i++) {
        pthread_join(threads[i], NULL);
    }

    return 0;
}
```

Explicação do Código

1. **Linha** `(void *)(intptr_t)i`: Converte int para void*
 - `intptr_t` é tipo que cabe um ponteiro e um int
 - Garante portabilidade em 32/64 bits
2. **Linha na** `tarefa()`: Converte de volta
 - `(int)(intptr_t)arg` reverte a conversão

Saída Esperada

```
Thread ID: 0
Thread ID: 1
Thread ID: 2
Thread ID: 3
Thread ID: 4
```

Problemas

- **Portabilidade:** Nem sempre um int cabe em um ponteiro
- **Valores negativos:** Podem causar problemas em algumas plataformas
- **Valores grandes:** Não funciona com long long ou tipos grandes

Inclusão Necessária

```
#include <stdint.h> /* Para intptr_t */
```

Técnica 2: Alocar Memória (malloc)

Quando Usar

Quando passar valor que não cabe em ponteiro ou múltiplos valores.

Funcionamento

1. Alocar memória no heap
2. Copiar valor para memória
3. Passar endereço para thread
4. Thread libera memória

Exemplo: Passagem de Inteiro com malloc

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

void *tarefa(void *arg) {
    int *id_ptr = (int *)arg;
    int id = *id_ptr;
    printf("Thread ID: %d\n", id);

    free(id_ptr); /* Libera memória alocada */
    return NULL;
}

int main() {
    pthread_t threads[5];

    for (int i = 0; i < 5; i++) {
        int *id_ptr = malloc(sizeof(int)); /* Aloca */
        *id_ptr = i;
        pthread_create(&threads[i], NULL, tarefa, id_ptr);
    }

    for (int i = 0; i < 5; i++) {
        pthread_join(threads[i], NULL);
    }

    return 0;
}
```

Explicação do Código

1. **Linha malloc:** Aloca espaço para um int
2. ***_Linha _id_ptr = i:___** Copia valor para memória alocada
3. **Linha pthread_create:** Passa endereço da memória

4. **Na tarefa():** Desreferencia para obter valor
5. **Linha free:** Libera memória na thread

Saída Esperada

```
Thread ID: 0  
Thread ID: 1  
Thread ID: 2  
Thread ID: 3  
Thread ID: 4
```

Vantagens

- Funciona com qualquer tipo
- Permite múltiplos valores
- Mais previsível que casting

Cuidados

- **Memory leak:** Não esquecer free()
- **Thread lenta:** Se thread for lenta, malloc ainda é válido
- **Sincronização:** Cada thread tem seu próprio malloc, sem race

Técnica 3: Usar Struct (Múltiplos Valores)

Quando Usar

Para passar múltiplos valores de tipos diferentes.

Funcionamento

1. Define struct com todos os campos
2. Aloca memória para struct
3. Preenche campos
4. Passa ponteiro da struct
5. Thread acessa campos via ponteiro

Exemplo: Struct com Múltiplos Dados

```
#include <pthread.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>
```

```

typedef struct {
    int id;
    char nome[50];
    int valor;
} thread_args_t;

void *tarefa(void *arg) {
    thread_args_t *args = (thread_args_t *)arg;

    printf("Thread %d: Nome=%s, Valor=%d\n",
        args->id, args->nome, args->valor);

    free(args); /* Libera struct */
    return NULL;
}

int main() {
    pthread_t threads[3];

    for (int i = 0; i < 3; i++) {
        thread_args_t *args = malloc(sizeof(thread_args_t));

        args->id = i;
        sprintf(args->nome, "Thread_%d", i);
        args->valor = i * 100;

        pthread_create(&threads[i], NULL, tarefa, args);
    }

    for (int i = 0; i < 3; i++) {
        pthread_join(threads[i], NULL);
    }

    return 0;
}

```

Explicação do Código

1. **typedef struct:** Define tipo com múltiplos campos
2. **malloc(sizeof(thread_args_t)):** Aloca memória para struct
3. **args->id = i:** Preenche campo id
4. **pthread_create(..., args):** Passa struct
5. **Na tarefa():** Acessa campos via `args->campo`
6. **free(args):** Libera struct inteira

Saída Esperada

```
Thread 0: Nome=Thread_0, Valor=0
Thread 1: Nome=Thread_1, Valor=100
Thread 2: Nome=Thread_2, Valor=200
```

Vantagens

- Organizado e legível
- Suporta qualquer tipo de dado
- Escalável para muitos valores
- Usa stack da thread para variáveis locais

Padrão Recomendado

```
typedef struct {
    int id;
    int inicio;
    int fim;
    float *dados;
    /* ... outros campos */
} worker_t;

void *worker_func(void *arg) {
    worker_t *w = (worker_t *)arg;
    /* Usar w->id, w->inicio, etc. */
    free(w);
    return NULL;
}
```

Técnica 4: Usar Array Estático (Sem Malloc)

Quando Usar

Quando número de threads é **fixo e pequeno** e não quer malloc.

Funcionamento

Declarar array de structs estático, preencher, passar endereço.

Exemplo

```
#include <pthread.h>
#include <stdio.h>

typedef struct {
```

```

    int id;
    char mensagem[100];
} Args;

void *tarefa(void *arg) {
    Args *a = (Args *)arg;
    printf("ID: %d, Mensagem: %s\n", a->id, a->mensagem);
    return NULL;
}

int main() {
    #define N 3
    Args args[N]; /* Array estático */
    pthread_t threads[N];

    /* Preencher array */
    for (int i = 0; i < N; i++) {
        args[i].id = i;
        sprintf(args[i].mensagem, "Olá %d", i);
    }

    /* Criar threads */
    for (int i = 0; i < N; i++) {
        pthread_create(&threads[i], NULL, tarefa, &args[i]);
    }

    /* Join */
    for (int i = 0; i < N; i++) {
        pthread_join(threads[i], NULL);
    }

    return 0;
}

```

Vantagens

- Sem malloc/free
- Simples para número pequeno de threads
- Sem memory leak

Problemas

- Número de threads deve ser conhecido em compile-time
 - Não funciona para threads dinâmicas
 - Stack pode ter limite
-

Comparação de Técnicas

Técnica	Simplidade	Flexibilidade	Segurança	Caso de Uso
Casting	Muito alta	Baixa	Baixa	Um int pequeno
malloc inteiro	Alta	Média	Média	Um valor simples
Struct + malloc	Média	Alta	Alta	Múltiplos valores (RECOMENDADO)
Array estático	Alta	Baixa	Média	Número fixo de threads

Exemplo Completo: Paralelizar Soma de Vetor

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define N 1000
#define NUM_THREADS 4

float vetor[N];
float soma_parcial[NUM_THREADS];

typedef struct {
    int id;
    int inicio;
    int fim;
} thread_args_t;

void *somar(void *arg) {
    thread_args_t *args = (thread_args_t *)arg;

    float soma = 0;
    for (int i = args->inicio; i < args->fim; i++) {
        soma += vetor[i];
    }

    soma_parcial[args->id] = soma;
    printf("Thread %d: Somou elementos [%d, %d] = %f\n",
        args->id, args->inicio, args->fim, soma);

    free(args);
}
```

```

    return NULL;
}

int main() {
    /* Inicializar vetor */
    for (int i = 0; i < N; i++) vetor[i] = 1.0;

    pthread_t threads[NUM_THREADS];
    int chunk = N / NUM_THREADS;

    /* Criar threads */
    for (int i = 0; i < NUM_THREADS; i++) {
        thread_args_t *args = malloc(sizeof(thread_args_t));
        args->id = i;
        args->inicio = i * chunk;
        args->fim = (i == NUM_THREADS - 1) ? N : (i + 1) * chunk;

        pthread_create(&threads[i], NULL, somar, args);
    }

    /* Aguardar threads */
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    /* Combinar resultados */
    float soma_total = 0;
    for (int i = 0; i < NUM_THREADS; i++) {
        soma_total += soma_parcial[i];
    }

    printf("\nSoma total: %f (esperado: %f)\n", soma_total, (float)N);

    return 0;
}

```

Saída Esperada

```

Thread 0: Somou elementos [0, 250] = 250.000000
Thread 1: Somou elementos [250, 500] = 250.000000
Thread 2: Somou elementos [500, 750] = 250.000000
Thread 3: Somou elementos [750, 1000] = 250.000000

Soma total: 1000.000000 (esperado: 1000.000000)

```

1. Use struct para múltiplos valores

```
/* BOM */
typedef struct { int a; int b; char c; } args_t;

/* EVITAR */
void *arg1, *arg2, *arg3; /* Impossível passar todos */
```

2. Sempre liberar malloc na thread

```
void *tarefa(void *arg) {
    /* ... usar arg */
    free(arg); /* Sempre! */
    return NULL;
}
```

3. Se usar casting, inclua stdint.h

```
#include <stdint.h>
pthread_create(&t, NULL, f, (void *)(intptr_t)valor);
```

4. Não passe endereço de variável local

```
/* ERRADO */
int x = 5;
pthread_create(&t, NULL, f, &x); /* &x pode mudar! */

/* CERTO */
int *x = malloc(sizeof(int));
*x = 5;
pthread_create(&t, NULL, f, x);
```

5. Use array estático só para número fixo

```
Args args[5]; /* OK se sempre 5 threads */
for (int i = 0; i < 5; i++) pthread_create(&t[i], NULL, f, &args[i]);
```

Erros Frequentes em Provas

- **Passar &i em loop:** Todas as threads veem valor final
 - **Não liberar malloc:** Memory leak
 - **Cast incorreto intptr_t:** Usar apenas para int, não struct
 - **Misturar técnicas:** Alocar e não liberar depois
 - **Esquecer * ao desreferenciar:** arg vs. *arg
 - **Struct não inicializada:** Acessar campo com lixo
 - **Não sincronizar acesso a dados compartilhados:** Race condition fora da thread
-

Seção Crítica e Mutex

Seção Crítica

Definição

Trecho de código que acessa recurso compartilhado que pode ser modificado por múltiplas threads.

Objetivo

Identificar e proteger contra race conditions.

Exemplo de Código Sem Proteção

```
int contador = 0; /* Compartilhado */

void *incrementar(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        contador++; /* SEÇÃO CRÍTICA */
    }
    return NULL;
}
```

Problema: Se 2 threads executarem, esperamos `contador = 2000000`, mas pode ser menor (race condition).

Por que Ocorre?

```
contador++;
/* Traduz-se em: */
/* 1. Carregar contador no registrador (LOAD) */
/* 2. Incrementar registrador (ADD) */
/* 3. Armazenar de volta em memória (STORE) */
/* Se threads se intercalam durante esses 3 passos, há problema! */
```

Mutex (Mutual Exclusion)

Definição

Mecanismo de sincronização que age como um "cadeado" para proteger seções críticas.

Objetivo

Garantir exclusão mútua: apenas uma thread por vez na seção crítica.

Funcionamento

- **Lock:** Obtém cadeado (bloqueia se não disponível)
- **Seção Crítica:** Executa código protegido
- **Unlock:** Libera cadeado

Sintaxe

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex; // variável de lock para exclusão mútua

int pthread_mutex_init(&mutex, NULL);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Exemplo Completo

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define N_THREADS 4
#define N_INCREMENTOS 1000000

int contador = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *incrementar(void *arg) {
    for (int i = 0; i < N_INCREMENTOS; i++) {
        pthread_mutex_lock(&mutex);
        contador++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main() {
    pthread_t threads[N_THREADS];

    for (int i = 0; i < N_THREADS; i++) {
        pthread_create(&threads[i], NULL, incrementar, NULL);
    }

    for (int i = 0; i < N_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Contador final: %d (esperado: %d)\n",
```

```
        contador, N_THREADS * N_INCREMENTOS);

    pthread_mutex_destroy(&mutex);
    return 0;
}
```

Explicação do Código

1. **Linha mutex global:** Inicializa mutex com estado padrão
2. **Linha lock:** Obtém exclusão (bloqueia se outra thread estiver)
3. **Linha contador++:** Operação atômica agora (só 1 thread por vez)
4. **Linha unlock:** Libera cadeado para próxima thread
5. **Linha destroy:** Libera recursos do mutex ao final

Saída Esperada

Contador final: 4000000 (esperado: 4000000)

Problemas Comuns

- **Deadlock:** Thread tenta pegar mutex que já possui (sem ser recursivo)
- **Esquecer unlock:** Outras threads ficam bloqueadas para sempre
- **Inconsistência:** Esquecer lock em uma das operações críticas

Boas Práticas

1. **Minimize a seção crítica:**

```
/* BOM: Cálculo fora do lock */
int valor_temp = calculo_pesado();
pthread_mutex_lock(&mutex);
variavel_compartilhada += valor_temp;
pthread_mutex_unlock(&mutex);

/* RUIM: Cálculo dentro do lock */
pthread_mutex_lock(&mutex);
variavel_compartilhada += calculo_pesado();
pthread_mutex_unlock(&mutex);
```

2. **Sempre destruir mutex**
3. **Preferir variáveis locais**

Semáforos

Definição

Primitiva de sincronização baseada em contador inteiro com operações atômicas.

Tipos

Tipo	Descrição
Binário	Valor 0 ou 1 (similar a mutex)
Contador	Valor 0 a N (controla múltiplas instâncias)
Nomeado	Global ao sistema (processos independentes)
Não Nomeado	Na memória do processo (threads)

Operações

- **wait(sem):** Decrementa; se 0, bloqueia
- **post(sem):** Incrementa; acorda thread se houver

Sintaxe

```
#include <semaphore.h>

int sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_wait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_destroy(sem_t *sem);
```

Exemplo: Controle de Recursos (Estacionamento)

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>

#define N_CARROS 5
#define VAGAS 2

sem_t vagas;

void *carro(void *arg) {
    int id = *(int *)arg;

    printf("Carro %d: tentando entrar...\n", id);
    sem_wait(&vagas); /* Tenta pegar uma vaga */
```

```

    printf("Carro %d: ESTACIONADO.\n", id);
    sleep(2); /* Simula permanência */

    printf("Carro %d: SAINDO...\n", id);
    sem_post(&vagas); /* Libera vaga */

    free(arg);
    return NULL;
}

int main() {
    pthread_t threads[N_CARROS];

    sem_init(&vagas, 0, VAGAS); /* 2 vagas disponíveis */

    for (int i = 0; i < N_CARROS; i++) {
        int *id = malloc(sizeof(int));
        *id = i + 1;
        pthread_create(&threads[i], NULL, carro, id);
    }

    for (int i = 0; i < N_CARROS; i++) {
        pthread_join(threads[i], NULL);
    }

    sem_destroy(&vagas);
    printf("Simulação completada.\n");
    return 0;
}

```

Saída Esperada (Exemplo)

```

Carro 1: tentando entrar...
Carro 1: ESTACIONADO.
Carro 2: tentando entrar...
Carro 2: ESTACIONADO.
Carro 3: tentando entrar...
Carro 1: SAINDO...
Carro 3: ESTACIONADO.
...

```

Diferença entre Mutex e Semáforo

Aspecto	Mutex	Semáforo
Instâncias	1 por vez	N por vez

Aspecto	Mutex	Semáforo
Conceito de Dono	Sim	Não
Uso Típico	Exclusão	Recursos

Variáveis de Condição

Definição

Mecanismo que permite thread suspender até que condição lógica se torne verdadeira.

Objetivo

Evitar "espera ocupada" (busy waiting): em vez de testar repetidamente uma condição, a thread se bloqueia e libera CPU.

Funcionamento

- **Trabalham com Mutex:** Protegem a variável que representa a condição
- **wait:** Libera mutex ATOMICAMENTE e bloqueia
- **signal:** Acorda uma thread
- **broadcast:** Acorda TODAS as threads

Sintaxe

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Exemplo: Produtor e Consumidor

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

int dado_pronto = 0;
int dado = 0;

void *produtor(void *arg) {
```

```

    sleep(2); /* Simula processamento */

    pthread_mutex_lock(&m);
    dado = 42;
    dado_pronto = 1;
    printf("Produtor: Dado pronto = %d\n", dado);
    pthread_cond_signal(&c); /* Acorda consumidor */
    pthread_mutex_unlock(&m);

    return NULL;
}

void *consumidor(void *arg) {
    pthread_mutex_lock(&m);

    /* SEMPRE use while, não if! */
    while (!dado_pronto) {
        printf("Consumidor: Aguardando...\n");
        pthread_cond_wait(&c, &m); /* Libera m e bloqueia */
    }

    printf("Consumidor: Recebeu %d\n", dado);
    pthread_mutex_unlock(&m);

    return NULL;
}

int main() {
    pthread_t tp, tc;

    pthread_create(&tc, NULL, consumidor, NULL);
    pthread_create(&tp, NULL, produtor, NULL);

    pthread_join(tp, NULL);
    pthread_join(tc, NULL);

    return 0;
}

```

Saída Esperada

```

Consumidor: Aguardando...
Produtor: Dado pronto = 42
Consumidor: Recebeu 42

```

Por que usar `while` não `if` ?

```
/* ERRADO */
if (!dado_pronto)
    pthread_cond_wait(&c, &m);
printf("Consumidor: Recebeu %d\n", dado); /* Pode estar falso! */

/* CORRETO */
while (!dado_pronto)
    pthread_cond_wait(&c, &m);
printf("Consumidor: Recebeu %d\n", dado); /* Garantidamente verdadeiro */
```

Motivo: Despertar espúrio (spurious wakeup) ou múltiplos consumidores podem acordar sem a condição ser verdadeira.

Barreiras

Definição

Ponto de sincronização onde todas as threads de um grupo DEVEM chegar antes que qualquer uma continue.

Objetivo

Coordenar algoritmos que operam em fases (etapas).

Sintaxe

```
int pthread_barrier_init(pthread_barrier_t *barrier,
                        const pthread_barrierattr_t *attr,
                        unsigned count);
int pthread_barrier_wait(pthread_barrier_t *barrier);
int pthread_barrier_destroy(pthread_barrier_t *barrier);
```

Exemplo: Algoritmo de Jacobi

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define N 4
#define NTHREADS 2

double x[N], x_new[N];
pthread_barrier_t barreira;

typedef struct { int id; int start; int end; } thread_data_t;
```

```

void *jacobi_thread(void *arg) {
    thread_data_t *data = (thread_data_t *)arg;

    for (int iteracao = 0; iteracao < 3; iteracao++) {
        /* Fase 1: Cálculo */
        for (int i = data->start; i < data->end; i++) {
            x_new[i] = x[i] + 1.0; /* Cálculo simplificado */
        }

        printf("Thread %d: Calculou fase 1\n", data->id);
        pthread_barrier_wait(&barreira); /* Sincroniza */

        /* Fase 2: Atualização */
        for (int i = data->start; i < data->end; i++) {
            x[i] = x_new[i];
        }

        printf("Thread %d: Atualizou fase 2\n", data->id);
        pthread_barrier_wait(&barreira); /* Sincroniza novamente */
    }

    return NULL;
}

int main() {
    pthread_t threads[NTHREADS];
    thread_data_t data[NTHREADS];

    /* Inicializa array */
    for (int i = 0; i < N; i++) x[i] = 1.0;

    /* Cria barreira para NTHREADS threads */
    pthread_barrier_init(&barreira, NULL, NTHREADS);

    int chunk = N / NTHREADS;
    for (int t = 0; t < NTHREADS; t++) {
        data[t].id = t;
        data[t].start = t * chunk;
        data[t].end = (t == NTHREADS - 1) ? N : (t + 1) * chunk;
        pthread_create(&threads[t], NULL, jacobi_thread, &data[t]);
    }

    for (int t = 0; t < NTHREADS; t++) {
        pthread_join(threads[t], NULL);
    }

    printf("Resultado final: x[0] = %f\n", x[0]);
    pthread_barrier_destroy(&barreira);
}

```

```
    return 0;
}
```

Condição de Corrida (Race Condition)

Definição

Situação onde múltiplas threads acessam mesmo dado compartilhado e pelo menos uma faz escrita, resultando em valor imprevisível.

Objetivo de Compreender

Identificar bugs concorrentes e como evitá-los.

Exemplo Clássico

```
int contador = 0; /* Compartilhado */

void *incrementar(void *arg) {
    for (int i = 0; i < 100000; i++) {
        contador++; /* RACE CONDITION */
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, incrementar, NULL);
    pthread_create(&t2, NULL, incrementar, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Contador = %d (esperado 200000)\n", contador);
    /* Saída típica: 100250, 153472, etc. (NUNCA 200000) */
    return 0;
}
```

Por que Ocorre?

Instrução em nível de máquina:
contador++;

Traduz-se em:

1. LOAD contador → registrador
2. ADD 1 → registrador

3. STORE registrador → contador

Entrelace possível:

T1: LOAD contador (valor 5) → registrador1

T2: LOAD contador (valor 5) → registrador2

T1: ADD 1 → registrador1 (resultado 6)

T2: ADD 1 → registrador2 (resultado 6)

T1: STORE 6 → contador

T2: STORE 6 → contador

Resultado: contador = 6 (deveria ser 7!)

Solução

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *incrementar(void *arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&mutex);
        contador++; /* Agora atômico */
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

Identificação

- Resultado varia a cada execução
- Falta de sincronização em acesso compartilhado
- Pelo menos uma operação é escrita

Deadlock

Definição

Situação onde 2+ threads ficam bloqueadas para sempre, cada uma esperando por recurso que a outra possui.

Condições Necessárias

1. **Exclusão Mútua:** Recurso é exclusivo
2. **Retenção:** Thread retém recurso enquanto aguarda
3. **Não-preempção:** Não há tirado de thread
4. **Espera Circular:** $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$

Exemplo Clássico

```
pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1_func(void *arg) {
    pthread_mutex_lock(&m1);
    printf("T1: Tem m1\n");
    sleep(1); /* Garante que T2 vai tentar m2 */

    printf("T1: Tentando m2...\n");
    pthread_mutex_lock(&m2); /* DEADLOCK: T2 tem m2! */
    printf("T1: Tem m2\n");

    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);
    return NULL;
}

void *thread2_func(void *arg) {
    pthread_mutex_lock(&m2);
    printf("T2: Tem m2\n");
    sleep(1);

    printf("T2: Tentando m1...\n");
    pthread_mutex_lock(&m1); /* DEADLOCK: T1 tem m1! */
    printf("T2: Tem m1\n");

    pthread_mutex_unlock(&m1);
    pthread_mutex_unlock(&m2);
    return NULL;
}
```

Resultado: Ambas as threads ficam bloqueadas esperando infinitamente.

Solução: Ordem Consistente

```
void *thread1_func(void *arg) {
    pthread_mutex_lock(&m1);
    pthread_mutex_lock(&m2);
    /* ... uso */
    pthread_mutex_unlock(&m2);
    pthread_mutex_unlock(&m1);
}

void *thread2_func(void *arg) {
    pthread_mutex_lock(&m1); /* MESMA ORDEM */
    pthread_mutex_lock(&m2);
```

```
/* ... uso */
pthread_mutex_unlock(&m2);
pthread_mutex_unlock(&m1);
}
```

Detecção

- Programa "trava" sem erro
- Threads ficam em estado "WAITING"
- Nenhuma saída/progresso

Prevenção

1. **Ordem consistente:** Sempre adquirir locks na mesma ordem
 2. **Timeouts:** Usar `pthread_mutex_trylock` com timeout
 3. **Lock-free:** Usar estruturas atômicas
 4. **Timeout em operações I/O**
-

Starvation (Inanição)

Definição

Starvation (inanição) é a condição em que uma thread ou processo é sistematicamente preterido no acesso a um recurso compartilhado, ficando indefinidamente impedido de progredir — mesmo que o sistema como um todo continue operando. Diferente de deadlock, não há bloqueio mútuo: as demais threads avançam, mas uma delas praticamente nunca é atendida.

Objetivo

Compreender starvation é fundamental para projetar sistemas concorrentes **justos** (*fair*): sistemas onde todas as threads têm garantia de progresso dentro de um intervalo razoável de tempo.

Funcionamento

Comportamento na memória e no escalonamento

A starvation não corrompe dados diretamente. Ela emerge de **políticas de escalonamento ou de sincronização que privilegiam repetidamente as mesmas threads**:

- **Escalonamento por prioridade estática:** Threads de baixa prioridade podem nunca ser escolhidas se threads de alta prioridade estão sempre prontas.

- **Mutex não-justo (padrão POSIX):** O padrão `pthread_mutex_t` não garante ordem FIFO. Uma thread pode ser acordada repetidamente enquanto outra espera na fila indefinidamente.
- **Semáforos:** `sem_post` acorda uma thread arbitrária; não há garantia de ordem.
- **Leitores-Escritores com prioridade de leitores:** Se há sempre novos leitores chegando, escritores podem ficar esperando indefinidamente.

Risco de progressão zero

```
Thread A (alta prioridade): → CPU → CPU → CPU → CPU → CPU → ...
Thread B (alta prioridade): → CPU → CPU → CPU → CPU → CPU → ...
Thread C (baixa prioridade):                (nunca recebe CPU)
```

Diferença para Deadlock e Livelock

Fenômeno	Progresso global	Quem está preso
Deadlock	Nenhum	Todas as threads envolvidas
Starvation	Sim (parcial)	Uma ou poucas threads
Livelock	Nenhum real	Todas (trocam estado mas não avançam)

Sintaxe Relacionada

```
/* Mutex com atributo de protocolo para mitigar starvation por inversão de
prioridade */
pthread_mutexattr_t attr;
pthread_mutexattr_init(&attr);
pthread_mutexattr_setprotocol(&attr, PTHREAD_PRIO_INHERIT);

pthread_mutex_t mutex;
pthread_mutex_init(&mutex, &attr);

/* Verificar se lock está disponível sem bloquear */
int pthread_mutex_trylock(pthread_mutex_t *mutex);

/* Semáforo com valor explícito para controle manual de justiça */
sem_t sem;
sem_init(&sem, 0, valor_inicial);
```

Parâmetros Relevantes

Função/Atributo	Parâmetro	Significado
pthread_mutexattr_setprotocol	PTHREAD_PRIO_INHERIT	Herda prioridade do esperador mais urgente
pthread_mutexattr_setprotocol	PTHREAD_PRIO_PROTECT	Usa prioridade máxima predefinida
sem_init	value	Número de acessos simultâneos permitidos
pthread_mutex_trylock	—	Retorna EBUSY sem bloquear (útil para implementar backoff)

Valor de Retorno

```
pthread_mutex_trylock(&mutex);
/* Retorna: 0 (sucesso), EBUSY (mutex já ocupado), ou código de erro */
```

Exemplo Completo

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Demonstra starvation em cenário de leitores-escritores sem controle justo */
/* Escritor morre de fome enquanto leitores chegam continuamente */

#define MAX_LEITORES 5
#define ITERACOES 6

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t sem_leitura = PTHREAD_COND_INITIALIZER;
pthread_cond_t sem_escrita = PTHREAD_COND_INITIALIZER;

int num_leitores = 0; /* Leitores ativos agora */
int leitores_esperando = 0; /* Aguardando para entrar */
int escritor_presente = 0; /* Escritor dentro da seção crítica */
int escritor_esperando = 0; /* Escritor aguardando */

/* -----
 * Versão SEM proteção a starvation: prioridade para leitores
 * ----- */
void leitura_sem_fairness_inicio(int id) {
    pthread_mutex_lock(&mutex);
```

```

/* Leitor entra se não há escritor presente */
while (escritor_presente) {
    leitores_esperando++;
    pthread_cond_wait(&sem_leitura, &mutex);
    leitores_esperando--;
}

num_leitores++;
printf("Leitor  %d: LENDO  (ativos: %d)\n", id, num_leitores);
pthread_mutex_unlock(&mutex);
}

void leitura_sem_fairness_fim(int id) {
    pthread_mutex_lock(&mutex);
    num_leitores--;
    printf("Leitor  %d: saiu   (ativos: %d)\n", id, num_leitores);

    if (num_leitores == 0) {
        pthread_cond_signal(&sem_escrita); /* Acorda escritor */
    }

    pthread_mutex_unlock(&mutex);
}

/* -----
 * Versão COM proteção a starvation: escritor tem prioridade
 * quando está esperando (aging manual)
 * ----- */
void leitura_com_fairness_inicio(int id) {
    pthread_mutex_lock(&mutex);

    /* Leitor espera se há escritor ativo OU esperando */
    while (escritor_presente || escritor_esperando > 0) {
        leitores_esperando++;
        pthread_cond_wait(&sem_leitura, &mutex);
        leitores_esperando--;
    }

    num_leitores++;
    printf("Leitor  %d: LENDO  (ativos: %d)\n", id, num_leitores);
    pthread_mutex_unlock(&mutex);
}

void leitura_com_fairness_fim(int id) {
    pthread_mutex_lock(&mutex);
    num_leitores--;
    printf("Leitor  %d: saiu   (ativos: %d)\n", id, num_leitores);

    if (num_leitores == 0) {
        pthread_cond_signal(&sem_escrita);
    }
}

```

```

    }

    pthread_mutex_unlock(&mutex);
}

void escrita_inicio(int id) {
    pthread_mutex_lock(&mutex);

    escritor_esperando++;
    printf("Escritor %d: aguardando acesso exclusivo...\n", id);

    /* Aguarda até não haver leitores nem outro escritor */
    while (num_leitores > 0 || escritor_presente) {
        pthread_cond_wait(&sem_escrita, &mutex);
    }

    escritor_esperando--;
    escritor_presente = 1;
    printf("Escritor %d: ESCRIVENDO\n", id);
    pthread_mutex_unlock(&mutex);
}

void escrita_fim(int id) {
    pthread_mutex_lock(&mutex);
    escritor_presente = 0;
    printf("Escritor %d: terminou\n", id);

    /* Acorda escritores primeiro; só depois leitores */
    if (escritor_esperando > 0) {
        pthread_cond_signal(&sem_escrita);
    } else {
        pthread_cond_broadcast(&sem_leitura);
    }

    pthread_mutex_unlock(&mutex);
}

/* -----
 * Threads
 * ----- */
void *thread_leitor(void *arg) {
    int id = (int)(intptr_t)arg;

    for (int i = 0; i < ITERACOES; i++) {
        leitura_com_fairness_inicio(id); /* Troque pela versão sem fairness
para comparar */
        usleep(200000);                  /* Simula leitura: 0.2 s */
        leitura_com_fairness_fim(id);
        usleep(50000);                   /* Pausa breve antes de reler */
    }
}

```

```

        return NULL;
    }

    void *thread_escritor(void *arg) {
        int id = (int)(intptr_t)arg;

        for (int i = 0; i < 3; i++) {
            usleep(100000); /* Simula intervalo antes de escrever */
            escrita_inicio(id);
            usleep(300000); /* Simula escrita: 0.3 s */
            escrita_fim(id);
        }

        return NULL;
    }

    int main(void) {
        pthread_t leitores[MAX_LEITORES];
        pthread_t escritor;

        printf("=== Simulacao de Starvation (com protecao ao escritor)
        ===\n\n");

        /* Cria escritor antes dos leitores para demonstrar que ele tem chance
        */
        pthread_create(&escritor, NULL, thread_escritor, (void *) (intptr_t)1);

        for (int i = 0; i < MAX_LEITORES; i++) {
            pthread_create(&leitores[i], NULL, thread_leitor, (void *) (intptr_t)
            (i + 1));
        }

        for (int i = 0; i < MAX_LEITORES; i++) {
            pthread_join(leitores[i], NULL);
        }
        pthread_join(escritor, NULL);

        printf("\nSimulacao concluida.\n");

        pthread_mutex_destroy(&mutex);
        pthread_cond_destroy(&sem_leitura);
        pthread_cond_destroy(&sem_escrita);

        return 0;
    }

```

Explicação do Código

1. `escritor_esperando` : Contador que sinaliza presença de escritor aguardando. Leitores usam esse campo para ceder prioridade — isso é **aging manual**.
2. `leitura_com_fairness_inicio()` : Leitor bloqueia não apenas quando há escritor ativo, mas também quando há escritor esperando (`escritor_esperando > 0`). Isso interrompe o fluxo contínuo de leitores.
3. `escrita_inicio()` : Incrementa `escritor_esperando` antes de bloquear, garantindo visibilidade para os leitores que vierem depois.
4. `escrita_fim()` : Dá prioridade a escritores pendentes na hora de acordar (`pthread_cond_signal(&sem_escrita)`) antes de liberar leitores com `broadcast` .
5. `leitura_sem_fairness_inicio()` : Versão problemática — ignora `escritor_esperando` , permitindo que leitores entrem mesmo com escritor esperando.
6. `usleep()` : Simula trabalho real dentro e fora da seção crítica para tornar o comportamento visível na saída.
7. `pthread_cond_broadcast(&sem_leitura)` : Acorda todos os leitores quando o escritor termina; eles competem igualmente pelo mutex.

Saída Esperada

```
=== Simulacao de Starvation (com protecao ao escritor) ===
```

```
Escritor 1: aguardando acesso exclusivo...
```

```
Leitor 1: LENDO (ativos: 1)
```

```
Leitor 2: LENDO (ativos: 2)
```

```
Leitor 1: saiu (ativos: 1)
```

```
Leitor 2: saiu (ativos: 0)
```

```
Escritor 1: ESCRIVENDO
```

```
Escritor 1: terminou
```

```
Leitor 3: LENDO (ativos: 1)
```

```
Leitor 4: LENDO (ativos: 2)
```

```
...
```

```
Escritor 1: ESCRIVENDO
```

```
Escritor 1: terminou
```

```
Simulacao concluida.
```

Comparação: Se trocar `leitura_com_fairness_inicio` por `leitura_sem_fairness_inicio` , o escritor raramente ou nunca consegue acesso enquanto leitores chegam continuamente.

Possíveis Problemas

- **Starvation de escritores:** Ocorre quando leitores chegam em fluxo contínuo e o lock não respeita `escritor_esperando` .
- **Starvation de baixa prioridade em escalonamento:** Thread com `SCHED_OTHER` e `nice` alto nunca ganha CPU se há threads `SCHED_FIFO` sempre prontas.

- **Inversão de prioridade:** Thread de alta prioridade espera por mutex retido por thread de baixa prioridade — a de alta fica privada de CPU indiretamente.
- **Aging ausente:** Sem incremento progressivo de prioridade, threads de menor peso podem esperar indefinidamente.
- `pthread_cond_signal` **arbitrário:** Acorda uma thread qualquer; sem ordem FIFO, as mesmas threads podem ser acordadas repetidamente.

Boas Práticas

1. **Registre o tempo de espera:** Implemente aging explícito para qualquer fila de espera crítica.

```
/* Registrar timestamp de entrada na fila
*/clock_gettime(CLOCK_MONOTONIC, &entrada);/* Comparar no escalonador
para elevar prioridade */
```

2. **Use** `PTHREAD_PRIO_INHERIT` **em mutexes críticos** para mitigar inversão de prioridade.
3. **Prefira** `pthread_cond_broadcast` **a** `pthread_cond_signal` quando múltiplas threads podem ser elegíveis — evita que a mesma thread seja acordada repetidamente.
4. **Minimize o tempo na seção crítica** para reduzir a janela em que outros esperam.
5. **Em leitores-escretores, contabilize escritores esperando** antes de deixar novos leitores entrarem.
6. **Teste com** `valgrind --tool=helgrind` para detectar padrões de acesso injusto.
7. **Evite prioridades estáticas sem aging** em sistemas de longa execução.

Comparação com Execução Sequencial

Aspecto	Sequencial	Paralelo sem fairness	Paralelo com fairness
Progresso de todas as threads	N/A	Não garantido	Garantido
Throughput	Baixo	Alto	Alto
Latência de escritores	N/A	Pode ser infinita	Limitada e previsível
Complexidade	Mínima	Média	Maior

Resumo Rápido

Starvation

- └─ Causa: política injusta de escalonamento ou sincronização
- └─ Efeito: thread não progride, mas sistema segue funcionando
- └─ Diferença do Deadlock: outras threads avançam
- └─ Solução principal: aging + prioridade de escritores
- └─ Detecção: thread em `WAITING` por tempo anormalmente longo

Violação de Atomicidade

Definição

Violação de Atomicidade (*Atomicity Violation*) é um bug de concorrência que ocorre quando um trecho de código que **deveria ser executado como uma unidade indivisível** (atomicamente) é interrompido entre duas ou mais operações distintas por outra thread, quebrando o invariante lógico que as operações juntas deveriam garantir.

Em outras palavras: o programador assume que "verificar" e "usar" um dado acontecem sem interrupção, mas o SO pode escalonar outra thread entre essas duas operações.

Objetivo

Compreender violações de atomicidade é essencial para:

- Identificar **race conditions sutis** que não são óbvias como um simples `contador++` sem lock.
- Projetar seções críticas que agrupem corretamente todas as operações interdependentes.
- Distinguir violações de atomicidade de outras categorias de bugs concorrentes (deadlock, starvation, violação de ordem).

Funcionamento

Comportamento na memória

Uma violação de atomicidade envolve tipicamente um padrão **"check-then-act"** ou **"read-modify-write"** sem proteção adequada:

Thread A	Thread B
verifica condição (ok)	← intercalação aqui
age sobre o estado	modifica o estado
	← estado já inválido!

A condição verificada por A era verdadeira **no momento da verificação**, mas tornou-se falsa antes de A agir sobre ela.

Tipos comuns

Tipo	Padrão	Exemplo
Check-then-act	Verificar estado → agir com base nele	<code>if (ptr != NULL) ptr->campo = 1</code>

Tipo	Padrão	Exemplo
Read-modify-write	Ler valor → modificar → gravar	x = x + y com x e y compartilhados
Transação incompleta	Atualizar múltiplos campos relacionados	Atualizar saldo e histórico separadamente

Compartilhamento de recursos e riscos

- A violação exige pelo menos **um acesso de escrita** intercalado.
- Pode ocorrer mesmo com dados que parecem "simples" (ponteiros, flags booleanas).
- É especialmente perigosa com ponteiros: verificar `ptr != NULL` e depois usar `ptr->campo` são duas operações separadas na máquina.

Sintaxe

```
/* ERRADO: check e act sem proteção */
if (ptr != NULL) {           /* Verificação */
    valor = ptr->campo;       /* Uso - outra thread pode ter liberado ptr
aqui! */
}

/* CORRETO: check e act dentro do mesmo lock */
pthread_mutex_lock(&mutex);
if (ptr != NULL) {
    valor = ptr->campo;
}
pthread_mutex_unlock(&mutex);
```

Parâmetros

Elemento	Papel na proteção
<code>pthread_mutex_lock</code>	Garante que nenhuma outra thread entra na seção entre check e act
<code>pthread_mutex_unlock</code>	Libera o lock após toda a operação composta estar completa
Seção crítica	Deve englobar todas as operações que dependem do mesmo invariante

Valor de Retorno

As funções de mutex retornam:

- **0**: Sucesso
- **Código de erro (não-zero)**: Falha (EINVAL, EDEADLK, etc.)

Exemplo Completo

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Exemplo: Violação de atomicidade em acesso a ponteiro compartilhado */
/* Thread Leitora verifica ptr e usa ptr->valor */
/* Thread Liberadora libera ptr entre essas duas operações */

typedef struct {
    int valor;
} Dado;

Dado *ptr_global = NULL;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

/* -----
 * Versão com VIOLAÇÃO de atomicidade (bugada)
 * ----- */
void *leitora_bugada(void *arg) {
    /* Passo 1: Verifica ponteiro */
    if (ptr_global != NULL) { /* OK neste instante... */

        /* ← AQUI outra thread pode fazer free(ptr_global) e ptr_global =
        NULL */
        usleep(10000); /* Simula intercalação - em código real não existe
        este sleep */

        /* Passo 2: Usa ponteiro - PODE CAUSAR SEGFAULT! */
        printf("[BUGADA] Valor lido: %d\n", ptr_global->valor); /* Undefined
        behavior */
    }
    return NULL;
}

/* -----
 * Versão CORRETA: check e act dentro do mesmo lock
 * ----- */
void *leitora_correta(void *arg) {
    pthread_mutex_lock(&mutex);

    /* Verificação e uso são agora uma única operação atômica */
    if (ptr_global != NULL) {
        printf("[CORRETA] Valor lido: %d\n", ptr_global->valor);
    } else {
        printf("[CORRETA] Ponteiro nulo, nada a ler.\n");
    }
}
```

```

    pthread_mutex_unlock(&mutex);
    return NULL;
}

void *liberadora(void *arg) {
    usleep(5000); /* Simula atraso antes de liberar */

    pthread_mutex_lock(&mutex);

    if (ptr_global != NULL) {
        printf("[LIBERADORA] Liberando recurso...\n");
        free(ptr_global);
        ptr_global = NULL;
        printf("[LIBERADORA] Ponteiro liberado e zerado.\n");
    }

    pthread_mutex_unlock(&mutex);
    return NULL;
}

/* -----
 * Segundo exemplo: Violação em transação de múltiplos campos
 * ----- */
typedef struct {
    int saldo;
    int num_transacoes;
} Conta;

Conta conta = {1000, 0};
pthread_mutex_t mutex_conta = PTHREAD_MUTEX_INITIALIZER;

/* ERRADO: atualiza campos separadamente */
void debitar_bugado(int valor) {
    conta.saldo -= valor; /* Thread B pode ler entre aqui... */
    conta.num_transacoes++; /* ...e aqui, vendo estado inconsistente */
}

/* CORRETO: agrupa toda a transação no lock */
void debitar_correto(int valor) {
    pthread_mutex_lock(&mutex_conta);
    conta.saldo -= valor;
    conta.num_transacoes++;
    pthread_mutex_unlock(&mutex_conta);
}

void *thread_debito(void *arg) {
    int valor = (int)(intptr_t)arg;

    debitar_correto(valor);
}

```

```

        printf("Debitou %d | Saldo: %d | Transacoes: %d\n",
               valor, conta.saldo, conta.num_transacoes);

    return NULL;
}

/* -----
 * main
 * ----- */
int main(void) {
    /* --- Demonstração 1: check-then-act com ponteiro --- */
    printf("=== Demonstracao 1: check-then-act (versao CORRETA) ===\n");

    ptr_global = malloc(sizeof(Dado));
    ptr_global->valor = 42;

    pthread_t t_leit, t_lib;
    pthread_create(&t_lib, NULL, liberadora, NULL);
    pthread_create(&t_leit, NULL, leitora_correta, NULL);

    pthread_join(t_lib, NULL);
    pthread_join(t_leit, NULL);

    /* --- Demonstração 2: transação de múltiplos campos --- */
    printf("\n=== Demonstracao 2: transacao atomica de conta ===\n");

    pthread_t debitadores[4];
    int valores[4] = {100, 200, 50, 150};

    for (int i = 0; i < 4; i++) {
        pthread_create(&debitadores[i], NULL, thread_debito,
                      (void *)(&valores[i]));
    }

    for (int i = 0; i < 4; i++) {
        pthread_join(debitadores[i], NULL);
    }

    printf("\nSaldo final: %d | Total de transacoes: %d\n",
           conta.saldo, conta.num_transacoes);

    pthread_mutex_destroy(&mutex);
    pthread_mutex_destroy(&mutex_conta);
    return 0;
}

```

Explicação do Código

1. `leitora_bugada()` : Realiza `if (ptr_global != NULL)` e depois acessa `ptr_global->valor` em dois momentos distintos e desprotegidos. Entre eles, `liberadora` pode chamar `free(ptr_global)`, tornando o ponteiro inválido — undefined behavior clássico.
2. `usleep(10000)` **artificial**: Simula o intervalo que naturalmente existe em execução real entre a verificação e o uso; em código de produção esse intervalo existe mesmo sem `sleep`, pois o SO pode escalonar outra thread a qualquer instrução.
3. `leitora_correta()` : Agrupa verificação e uso dentro do mesmo `pthread_mutex_lock / unlock`, tornando o par uma operação atômica. A thread `liberadora` não pode modificar `ptr_global` enquanto a leitora está dentro do lock.
4. `liberadora()` : Também usa o mesmo mutex antes de fazer `free` e zerar o ponteiro — imprescindível para que a proteção seja bilateral.
5. `debitar_bugado()` : Modifica `saldo` e `num_transacoes` em operações separadas. Uma thread leitora pode observar `saldo` já decrementado mas `num_transacoes` ainda não incrementado — estado inconsistente.
6. `debitar_correto()` : Toda a transação (dois campos) está dentro de um único lock, garantindo que nenhuma thread veja estado intermediário.
7. `thread_debito()` : Usa `debitar_correto` e imprime estado após a transação, também dentro do contexto seguro.

Saída Esperada

```
=== Demonstracao 1: check-then-act (versao CORRETA) ===
[LIBERADORA] Liberando recurso...
[LIBERADORA] Ponteiro liberado e zerado.
[CORRETA] Ponteiro nulo, nada a ler.
```

```
=== Demonstracao 2: transacao atomica de conta ===
Debitou 100 | Saldo: 900 | Transacoes: 1
Debitou 200 | Saldo: 700 | Transacoes: 2
Debitou 50  | Saldo: 650 | Transacoes: 3
Debitou 150 | Saldo: 500 | Transacoes: 4

Saldo final: 500 | Total de transacoes: 4
```

A ordem dos débitos pode variar (concorrência), mas saldo e número de transações serão sempre consistentes entre si.

Possíveis Problemas

- **Segmentation Fault por ponteiro dangling**: `ptr` foi liberado entre a verificação `!= NULL` e o uso `ptr->campo`.
- **Estado de objeto inconsistente**: Dois campos logicamente vinculados (`saldo` + `histórico`) são atualizados em momentos diferentes, permitindo observação de estado

inválido.

- **Dificuldade de reprodução:** A janela de intercalação pode ser de nanossegundos, tornando o bug raro em testes mas catastrófico em produção sob carga.
- **Falsa sensação de segurança:** Cada operação individualmente "parece" segura, mas a combinação não é.
- **Double-checked locking incorreto:** Padrão de otimização que verifica condição fora e dentro do lock sem barreiras de memória adequadas.

Boas Práticas

1. **Identifique invariantes:** Liste quais variáveis devem ser consistentes entre si e proteja todas elas com o mesmo lock.

```
/* Invariante: saldo e historico devem refletir exatamente as mesmas transações */pthread_mutex_lock(&mutex);saldo -= valor;historico[n_trans++] = valor;pthread_mutex_unlock(&mutex);
```

2. **Nunca separe check e act em operações protegidas por locks distintos** — a proteção deve ser a mesma instância de mutex.
3. **Desconfie do padrão** `if (x != NULL) usar(x)` em código concorrente — quase sempre requer lock ao redor de ambas as linhas.
4. **Use** `pthread_mutex_trylock` **com cautela:** Pode introduzir race conditions se o fluxo alternativo não for cuidadosamente analisado.
5. **Prefira objetos imutáveis** quando possível — eliminam violações de atomicidade por definição.
6. **Ferramentas:** `helgrind` (Valgrind), `ThreadSanitizer` (`-fsanitize=thread`) detectam violações de atomicidade em tempo de execução.

```
gcc -fsanitize=thread -g programa.c -o programa -lpthread./programa
```

Comparação com Execução Sequencial

Aspecto	Sequencial	Paralelo sem atomicidade	Paralelo com atomicidade
Consistência de estado	Sempre garantida	Pode ser violada a qualquer momento	Garantida dentro do lock
Desempenho	Base	Maior throughput, mas bugs	Throughput alto e correto
Depuração	Simples	Muito difícil (não-determinístico)	Mais previsível

Resumo Rápido

Violação de Atomicidade

- └─ Causa: operações interdependentes sem lock compartilhado
- └─ Padrão típico: "check-then-act" ou "read-modify-write"
- └─ Efeito: estado inconsistente, segfault, dados corrompidos
- └─ Solução: englobar TODAS as operações dependentes no MESMO mutex
- └─ Detecção: ThreadSanitizer (-fsanitize=thread), helgrind
- └─ Diferença de Race Condition simples: envolve semântica de composição, não apenas acesso desprotegido a um único dado

Escalonamento

Definição

Processo pelo qual o SO decide qual thread/processo em estado "Pronto" ganha a CPU.

Algoritmos Comuns

Algoritmo	Descrição	Problema
FCFS	Primeira que chega, primeira a usar	Filas longas se primeira tarefa for pesada
Round Robin (RR)	Cada uma recebe quantum de tempo	Pode gerar muitos context switches
SPN	Tarefa mais curta primeiro	Inanição de tarefas longas
Prioridade	Maior prioridade primeiro	Inanição; mitigado com envelhecimento (aging)

Tipos

Tipo	Característica
Preemptivo	SO pode interromper tarefa (fim quantum, prioridade)
Não-Preemptivo	Tarefa executa até terminar ou bloquear

Quantum e Context Switch

- **Quantum:** Fatia de tempo para cada tarefa (típico: 10-100ms)
- **Context Switch:** Custo de salvar/restaurar estado de thread (overhead)

Implicação para Programação Paralela

- Ordem de execução é **não-determinística**

- Resultado pode variar entre execuções
- Sincronização é crítica

Paralelismo de Dados

Definição

Múltiplas threads executam **a mesma operação** sobre dados diferentes simultaneamente.

Objetivo

Acelerar processamento de grandes volumes de dados.

Padrão de Execução

Dados: [1, 2, 3, 4, 5, 6, 7, 8]

Threads: 2

T1: [1, 2, 3, 4] → Soma local

T2: [5, 6, 7, 8] → Soma local

Resultado final: Combinar somas

Exemplo Completo: Soma de Vetor

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define N 1000000
#define P 4

float array[N];
float soma_parciais[P];
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
float soma_total = 0;

typedef struct { int id; int inicio; int fim; } thread_data_t;

void *soma_parcial(void *arg) {
    thread_data_t *data = (thread_data_t *)arg;
    float soma_local = 0;

    /* Cada thread soma sua própria porção SEM LOCK */
    for (int i = data->inicio; i < data->fim; i++) {
```

```

        soma_local += array[i];
    }

    /* Seção crítica: atualizar resultado global */
    pthread_mutex_lock(&mutex);
    soma_total += soma_local;
    pthread_mutex_unlock(&mutex);

    free(data);
    return NULL;
}

int main() {
    pthread_t threads[P];

    /* Inicializa array */
    for (int i = 0; i < N; i++) array[i] = 1.0;

    int chunk = N / P;
    for (int i = 0; i < P; i++) {
        thread_data_t *td = malloc(sizeof(thread_data_t));
        td->id = i;
        td->inicio = i * chunk;
        td->fim = (i == P - 1) ? N : (i + 1) * chunk;

        pthread_create(&threads[i], NULL, soma_parcial, td);
    }

    for (int i = 0; i < P; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Soma total = %f (esperado: %f)\n", soma_total, (float)N);
    pthread_mutex_destroy(&mutex);
    return 0;
}

```

Vantagens

- **Escalabilidade:** Adicionar processadores melhora performance
- **Simplicidade:** Cada thread executa mesmo código
- **Comunicação mínima:** Cada thread trabalha independentemente

Limitações

- **Balanceamento de Carga:** Dados podem ter processamento não-uniforme
 - **Overhead:** Se tarefas forem muito pequenas, overhead supera ganho
-

Paralelismo de Tarefas

Definição

Múltiplas threads executam **operações diferentes** (tarefas) simultaneamente.

Objetivo

Explorar paralelismo quando tarefas são naturalmente distintas.

Padrão de Execução

```
T1: Ler arquivo → Processar → Escrever resultado
T2: Ler arquivo → Processar → Escrever resultado
T3: (Diferente) → Validar → Reportar
```

Exemplo: Pipeline de Processamento

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TAMANHO_FILA 100

typedef struct {
    char dados[256];
    int processado;
} Item;

Item fila[TAMANHO_FILA];
int indice_fila = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int fim = 0;

void *produtor(void *arg) {
    for (int i = 0; i < 10; i++) {
        pthread_mutex_lock(&mutex);

        sprintf(fila[indice_fila].dados, "Item %d", i);
        fila[indice_fila].processado = 0;
        indice_fila++;

        printf("Produtor: Criou item %d\n", i);
        pthread_cond_signal(&cond);
        pthread_mutex_unlock(&mutex);
    }
}
```

```

    pthread_mutex_lock(&mutex);
    fim = 1;
    pthread_cond_broadcast(&cond);
    pthread_mutex_unlock(&mutex);

    return NULL;
}

void *consumidor(void *arg) {
    int meu_id = *(int *)arg;

    while (1) {
        pthread_mutex_lock(&mutex);

        while (indice_fila == 0 && !fim) {
            pthread_cond_wait(&cond, &mutex);
        }

        if (indice_fila == 0 && fim) {
            pthread_mutex_unlock(&mutex);
            break;
        }

        indice_fila--;
        Item item = fila[indice_fila];

        pthread_mutex_unlock(&mutex);

        printf("Consumidor %d: Processando '%s'\n", meu_id, item.dados);
    }

    free(arg);
    return NULL;
}

int main() {
    pthread_t t_prod, t_cons[2];
    int ids[2] = {1, 2};

    pthread_create(&t_prod, NULL, produtor, NULL);
    for (int i = 0; i < 2; i++) {
        pthread_create(&t_cons[i], NULL, consumidor, &ids[i]);
    }

    pthread_join(t_prod, NULL);
    for (int i = 0; i < 2; i++) {
        pthread_join(t_cons[i], NULL);
    }
}

```

```
    return 0;
}
```

Produtor-Consumidor

Definição

Padrão clássico onde **produtores** geram dados e **consumidores** os processam.

Objetivo

Desacoplar produção de consumo, permitindo diferentes velocidades.

Problemas a Resolver

1. **Buffer vazio:** Consumidor não pode ler
2. **Buffer cheio:** Produtor não pode escrever
3. **Sincronização:** Coordenar acesso

Exemplo Completo: Buffer Limitado

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define BUFFER_SIZE 5

typedef struct {
    int items[BUFFER_SIZE];
    int in; /* Posição para inserção */
    int out; /* Posição para extração */
    int count; /* Itens no buffer */
} Buffer;

Buffer buf = {.in = 0, .out = 0, .count = 0};
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t nao_vazio = PTHREAD_COND_INITIALIZER;
pthread_cond_t nao_cheio = PTHREAD_COND_INITIALIZER;

void *produtor(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < 10; i++) {
        int item = id * 100 + i;
```

```

pthread_mutex_lock(&mutex);

/* Aguarda buffer não estar cheio */
while (buf.count == BUFFER_SIZE) {
    printf("Produtor %d: Buffer cheio, aguardando...\n", id);
    pthread_cond_wait(&nao_cheio, &mutex);
}

/* Insere item */
buf.items[buf.in] = item;
buf.in = (buf.in + 1) % BUFFER_SIZE;
buf.count++;

printf("Produtor %d: Inseriu %d (count: %d)\n", id, item,
buf.count);

pthread_cond_signal(&nao_vazio);
pthread_mutex_unlock(&mutex);

usleep(100000); /* Simula produção */
}

free(arg);
return NULL;
}

void *consumidor(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < 10; i++) {
        pthread_mutex_lock(&mutex);

        /* Aguarda buffer não estar vazio */
        while (buf.count == 0) {
            printf("Consumidor %d: Buffer vazio, aguardando...\n", id);
            pthread_cond_wait(&nao_vazio, &mutex);
        }

        /* Extrai item */
        int item = buf.items[buf.out];
        buf.out = (buf.out + 1) % BUFFER_SIZE;
        buf.count--;

        printf("Consumidor %d: Consumiu %d (count: %d)\n", id, item,
buf.count);

        pthread_cond_signal(&nao_cheio);
        pthread_mutex_unlock(&mutex);

        usleep(200000); /* Simula consumo */
    }
}

```

```
}

free(arg);
return NULL;
}

int main() {
    pthread_t prod[2], cons[2];
    int ids[4] = {1, 2, 1, 2};

    for (int i = 0; i < 2; i++) {
        int *id = malloc(sizeof(int));
        *id = ids[i];
        pthread_create(&prod[i], NULL, produtor, id);
    }

    for (int i = 0; i < 2; i++) {
        int *id = malloc(sizeof(int));
        *id = ids[2 + i];
        pthread_create(&cons[i], NULL, consumidor, id);
    }

    for (int i = 0; i < 2; i++) {
        pthread_join(prod[i], NULL);
        pthread_join(cons[i], NULL);
    }

    return 0;
}
```

Leitores-Escritores

Definição

Múltiplos leitores podem acessar recurso **simultaneamente**, mas escritores requerem acesso **exclusivo**.

Objetivo

Maximizar concorrência em leitura sem comprometer consistência de escrita.

Restrições

1. Múltiplos leitores → OK
2. Um escritor → Exclui tudo
3. Escritor + leitor → Não permitido

Exemplo Completo

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define MAX_LEITORES 5
#define MAX_ESCRITORES 2

int recurso = 0; /* Dado compartilhado */
int num_leitores = 0;
int escritor_presente = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t podeder = PTHREAD_COND_INITIALIZER;
pthread_cond_t podedescrever = PTHREAD_COND_INITIALIZER;

void *leitor(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < 3; i++) {
        pthread_mutex_lock(&mutex);

        /* Aguarda nenhum escritor */
        while (escritor_presente) {
            printf("Leitor %d: Aguardando escritor sair...\n", id);
            pthread_cond_wait(&podeder, &mutex);
        }

        num_leitores++;
        printf("Leitor %d: LENDO (num_leitores: %d, recurso: %d)\n",
            id, num_leitores, recurso);

        pthread_mutex_unlock(&mutex);

        usleep(500000); /* Lê durante 0.5s */

        pthread_mutex_lock(&mutex);
        num_leitores--;

        if (num_leitores == 0) {
            pthread_cond_signal(&podedescrever);
        }

        printf("Leitor %d: Terminou leitura\n", id);
        pthread_mutex_unlock(&mutex);
    }
}
```

```

    free(arg);
    return NULL;
}

void *escritor(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < 2; i++) {
        pthread_mutex_lock(&mutex);

        /* Aguarda nenhum leitor e nenhum escritor */
        while (num_leitores > 0 || escritor_presente) {
            printf("Escritor %d: Aguardando acesso exclusivo...\n", id);
            pthread_cond_wait(&pode_escrever, &mutex);
        }

        escritor_presente = 1;
        printf("Escritor %d: ESCRIVENDO (recurso antigo: %d)\n", id,
recurso);

        pthread_mutex_unlock(&mutex);

        recurso = id * 10 + i; /* Modifica recurso */
        usleep(1000000); /* Escreve durante 1s */

        pthread_mutex_lock(&mutex);
        escritor_presente = 0;

        printf("Escritor %d: Terminou escrita (novo valor: %d)\n", id,
recurso);

        pthread_cond_broadcast(&pode_ler);
        pthread_cond_signal(&pode_escrever);
        pthread_mutex_unlock(&mutex);
    }

    free(arg);
    return NULL;
}

int main() {
    pthread_t leit[MAX_LEITORES], escrit[MAX_ESCRITORES];

    for (int i = 0; i < MAX_LEITORES; i++) {
        int *id = malloc(sizeof(int));
        *id = i + 1;
        pthread_create(&leit[i], NULL, leitor, id);
    }

    for (int i = 0; i < MAX_ESCRITORES; i++) {

```

```

    int *id = malloc(sizeof(int));
    *id = i + 1;
    pthread_create(&escrit[i], NULL, escritor, id);
}

for (int i = 0; i < MAX_LEITORES; i++) {
    pthread_join(leit[i], NULL);
}

for (int i = 0; i < MAX_ESCRITORES; i++) {
    pthread_join(escrit[i], NULL);
}

return 0;
}

```

Saída Esperada (Exemplo)

```

Leitor 1: LENDO (num_leitores: 1, recurso: 0)
Leitor 2: LENDO (num_leitores: 2, recurso: 0)
Leitor 3: LENDO (num_leitores: 3, recurso: 0)
Escritor 1: Aguardando acesso exclusivo...
Leitor 1: Terminou leitura
Leitor 2: Terminou leitura
Leitor 3: Terminou leitura
Escritor 1: ESCRIVENDO (recurso antigo: 0)
Escritor 1: Terminou escrita (novo valor: 10)
Leitor 4: LENDO (num_leitores: 1, recurso: 10)
...

```

Jantar dos Filósofos

Definição

Problema clássico de sincronização onde 5 filósofos alternam entre pensar e comer, compartilhando 5 garfos.

Objetivo

Evitar deadlock quando recursos são limitados e threads precisam de múltiplos recursos.

Restrições

- 5 filósofos em torno de uma mesa
- 5 garfos (um entre cada par de filósofos)

- Para comer, precisa de 2 garfos (esquerdo e direito)
- Pode-se pensar com 0 garfos

Problema: Deadlock Trivial

```
void *filosofo(void *arg) {
    int id = *(int *)arg;

    while (1) {
        printf("Filósofo %d: Pensando\n", id);
        sleep(1);

        /* PROBLEMA: Ordem não-coordenada */
        pthread_mutex_lock(&garfo[id]); /* Esquerdo */
        pthread_mutex_lock(&garfo[(id + 1) % 5]); /* Direito */

        printf("Filósofo %d: COMENDO\n", id);
        sleep(1);

        pthread_mutex_unlock(&garfo[id]);
        pthread_mutex_unlock(&garfo[(id + 1) % 5]);
    }
}
```

Deadlock: Todos pegam o garfo esquerdo e ficam esperando o direito!

Solução 1: Ordem Consistente

```
void *filosofo(void *arg) {
    int id = *(int *)arg;

    while (1) {
        printf("Filósofo %d: Pensando\n", id);
        sleep(1);

        int esquerdo = id;
        int direito = (id + 1) % 5;

        /* Ordena: sempre pega o menor primeiro */
        if (esquerdo > direito) {
            int temp = esquerdo;
            esquerdo = direito;
            direito = temp;
        }

        pthread_mutex_lock(&garfo[esquerdo]);
        pthread_mutex_lock(&garfo[direito]);
```

```

        printf("Filósofo %d: COMENDO\n", id);
        sleep(1);

        pthread_mutex_unlock(&garfo[direito]);
        pthread_mutex_unlock(&garfo[esquerdo]);
    }
}

```

Solução 2: Recurso Monitor

```

#define N_FILOSOFOS 5

enum { PENSANDO, COM_FOME, COMENDO } estado[N_FILOSOFOS];

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t pode_comer[N_FILOSOFOS];

void pega_garfos(int id) {
    pthread_mutex_lock(&mutex);

    estado[id] = COM_FOME;
    printf("Filósofo %d: COM FOME\n", id);

    int esquerdo = id;
    int direito = (id + 1) % N_FILOSOFOS;

    /* Aguarda até conseguir ambos os garfos */
    while (estado[esquerdo] == COMENDO ||
           estado[direito] == COMENDO) {
        printf("Filósofo %d: Aguardando garfos...\n", id);
        pthread_cond_wait(&pode_comer[id], &mutex);
    }

    estado[id] = COMENDO;
    printf("Filósofo %d: COMENDO\n", id);

    pthread_mutex_unlock(&mutex);
}

void larga_garfos(int id) {
    pthread_mutex_lock(&mutex);

    estado[id] = PENSANDO;
    printf("Filósofo %d: Voltou a pensar\n", id);

    int esquerdo = id;
    int direito = (id + 1) % N_FILOSOFOS;

    /* Notifica vizinhos que podem ter sido desbloqueados */
}

```

```

pthread_cond_signal(&pode_comer[esquerdo]);
pthread_cond_signal(&pode_comer[direito]);

pthread_mutex_unlock(&mutex);
}

void *filosofo(void *arg) {
    int id = *(int *)arg;

    for (int i = 0; i < 5; i++) {
        pega_garfos(id);
        sleep(1);
        larga_garfos(id);
        sleep(1);
    }

    free(arg);
    return NULL;
}

int main() {
    pthread_t threads[N_FILOSOFOS];

    for (int i = 0; i < N_FILOSOFOS; i++) {
        pthread_cond_init(&pode_comer[i], NULL);
    }

    for (int i = 0; i < N_FILOSOFOS; i++) {
        int *id = malloc(sizeof(int));
        *id = i;
        pthread_create(&threads[i], NULL, filosofo, id);
    }

    for (int i = 0; i < N_FILOSOFOS; i++) {
        pthread_join(threads[i], NULL);
    }

    return 0;
}

```

Saída Esperada (Exemplo)

```

Filósofo 0: Pensando
Filósofo 1: Pensando
Filósofo 2: Pensando
Filósofo 3: COM FOME
Filósofo 3: COMENDO
Filósofo 2: COM FOME
Filósofo 2: Aguardando garfos...

```

Filósofo 3: Voltou a pensar
Filósofo 2: COMENDO
...

Balanceamento de Carga

Definição

Balanceamento de carga (*load balancing*) é o conjunto de estratégias que distribui o trabalho entre threads (ou processos) de forma que todas permaneçam ocupadas durante o maior tempo possível, evitando que algumas fiquem ociosas enquanto outras acumulam trabalho.

O problema central é: **como dividir N unidades de trabalho entre P threads de modo que o tempo total de execução seja minimizado?**

Objetivo

- Maximizar a utilização de todos os núcleos disponíveis.
- Eliminar gargalos causados por threads desproporcionalmente sobrecarregadas.
- Reduzir o tempo total de execução (*makespan*) de uma aplicação paralela.
- Adaptar-se a variações no custo de cada tarefa (cargas heterogêneas).

Funcionamento

Tipos de balanceamento

Tipo	Quando a divisão ocorre	Custo de overhead	Adaptabilidade
Estático	Antes da execução	Mínimo	Nenhuma
Dinâmico	Durante a execução	Moderado	Alta
Work-stealing	Quando thread fica ociosa	Baixo (lazy)	Muito alta

Balanceamento estático

O trabalho é dividido igualmente no início. Ideal quando as tarefas têm custo **uniforme e previsível** (ex.: somar um vetor de N elementos com N/P elementos por thread).

```
Vetor: [1..1000000], P=4 threads
T1: [0,      250000)
T2: [250000, 500000)
T3: [500000, 750000)
T4: [750000, 1000000)
```

Problema: Se os custos forem heterogêneos (ex.: processar pixels com intensidades diferentes), algumas threads terminam cedo e ficam ociosas.

Balanceamento dinâmico

Threads pegam novas fatias de trabalho de uma fila compartilhada à medida que ficam ociosas. Threads rápidas ou com tarefas leves automaticamente compensam fazendo mais trabalho.

```
Fila compartilhada: [T1][T2][T3]...[T100]
Thread A: pega T1 → termina → pega T4 → ...
Thread B: pega T2 → termina → pega T5 → ...
Thread C: pega T3 (pesada) → ainda processando...
Thread A: pega T6, T7, T8 → compensando!
```

Work-stealing

Cada thread tem sua própria fila de tarefas. Quando uma thread esgota sua fila, ela "rouba" tarefas da fila de outra thread sobrecarregada. Minimiza contenção no caso comum (sem roubo).

Compartilhamento de recursos e sincronização

- **Estático:** Sem sincronização durante execução (fila predefinida por índice).
- **Dinâmico:** Mutex ou semáforo protege o contador/fila compartilhada.
- **Work-stealing:** Deque por thread com acesso duplo (dono pela frente, ladrão por trás); exige sincronização apenas no roubo.

Granularidade

O tamanho de cada fatia de trabalho impacta diretamente o trade-off:

- **Granularidade grossa:** Menos overhead de sincronização, mas pior balanceamento.
- **Granularidade fina:** Melhor balanceamento, mas alto custo de lock por tarefa.
- **Chunk adaptativo:** Começa com fatias grandes e reduz progressivamente (scheduling em loop).

Sintaxe

```
/* Padrão de balanceamento dinâmico com contador atômico compartilhado */
pthread_mutex_t mutex_fila = PTHREAD_MUTEX_INITIALIZER;
int proximo_item = 0; /* Índice da próxima tarefa disponível */
int total_itens = N;

/* Cada thread executa este loop */
while (1) {
```

```

pthread_mutex_lock(&mutex_fila);
int meu_item = proximo_item++; /* Pega próxima tarefa */
pthread_mutex_unlock(&mutex_fila);

if (meu_item >= total_itens) break; /* Sem mais trabalho */
processar(meu_item);
}

```

Parâmetros

Elemento	Papel
proximo_item	Índice global do próximo trabalho a ser retirado da fila
mutex_fila	Protege a leitura e incremento atômico de proximo_item
total_itens	Limite superior; thread para quando meu_item >= total_itens
chunk_size	Número de itens retirados de uma vez para reduzir contenção

Valor de Retorno

As funções de mutex e pthread retornam 0 em sucesso. O valor de negócio aqui é o speedup obtido:

```

Speedup ideal    = P  (número de threads)
Speedup real     = T_sequencial / T_paralelo
Eficiência       = Speedup / P  (ideal = 1.0)

```

Exemplo Completo

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>

/* -----
 * Balanceamento de carga: comparação entre estático e dinâmico
 * Cenário: processar N imagens com custos heterogêneos
 * ----- */

#define N_IMAGENS    20    /* Total de tarefas */
#define N_THREADS    4     /* Número de workers */
#define CHUNK_SIZE    2    /* Fatias no balanceamento dinâmico */

/* Custo simulado de cada imagem (ms) – propositalmente heterogêneo */
int custo_ms[N_IMAGENS] = {

```

```

    300, 10, 250, 20, 400, 15, 350, 30,
    500, 10, 100, 25, 450, 12, 200, 40,
    380, 18, 270, 22
};

/* --- Estruturas compartilhadas --- */
pthread_mutex_t mutex_dyn = PTHREAD_MUTEX_INITIALIZER;
int proximo_dyn = 0; /* Próximo índice livre (dinâmico) */

/* Tempo total consumido por cada thread (para análise de balanceamento) */
long tempo_estatico[N_THREADS];
long tempo_dinamico[N_THREADS];

/* -----
 * Utilitário: retorna tempo em milissegundos
 * ----- */
long agora_ms(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000L + ts.tv_nsec / 1000000L;
}

/* -----
 * Simula processamento de uma imagem
 * ----- */
void processar_imagem(int id_img, int id_thread, const char *modo) {
    usleep(custo_ms[id_img] * 1000); /* Simula custo real */
    printf("[%s] Thread %d processou imagem %2d (custo: %3d ms)\n",
           modo, id_thread, id_img, custo_ms[id_img]);
}

/* -----
 * BALANCEAMENTO ESTÁTICO
 * Cada thread processa um bloco fixo de N/P imagens
 * ----- */
typedef struct {
    int id;
    int inicio;
    int fim;
} args_estatico_t;

void *worker_estatico(void *arg) {
    args_estatico_t *a = (args_estatico_t *)arg;
    long t_inicio = agora_ms();

    for (int i = a->inicio; i < a->fim; i++) {
        processar_imagem(i, a->id, "ESTATICO");
    }

    tempo_estatico[a->id] = agora_ms() - t_inicio;
}

```

```

    printf("[ESTATICO] Thread %d terminou em %ld ms\n",
           a->id, tempo_estatico[a->id]);

    free(a);
    return NULL;
}

/* -----
 * BALANCEAMENTO DINÂMICO
 * Threads competem pelo próximo chunk disponível
 * ----- */
typedef struct {
    int id;
} args_dinamico_t;

void *worker_dinamico(void *arg) {
    args_dinamico_t *a = (args_dinamico_t *)arg;
    long t_inicio = agora_ms();

    while (1) {
        /* --- Seção crítica: pega próximo chunk --- */
        pthread_mutex_lock(&mutex_dyn);
        int meu_inicio = proximo_dyn;
        proximo_dyn += CHUNK_SIZE;
        pthread_mutex_unlock(&mutex_dyn);

        /* Sem mais trabalho? */
        if (meu_inicio >= N_IMAGENS) break;

        /* Processa até CHUNK_SIZE imagens (ou até o fim) */
        int meu_fim = meu_inicio + CHUNK_SIZE;
        if (meu_fim > N_IMAGENS) meu_fim = N_IMAGENS;

        for (int i = meu_inicio; i < meu_fim; i++) {
            processar_imagem(i, a->id, "DINAMICO");
        }
    }

    tempo_dinamico[a->id] = agora_ms() - t_inicio;
    printf("[DINAMICO] Thread %d terminou em %ld ms\n",
           a->id, tempo_dinamico[a->id]);

    free(a);
    return NULL;
}

/* -----
 * Análise de balanceamento: calcula desvio entre threads
 * ----- */
void analisar(const char *modo, long *tempos, int n) {

```

```

    long soma = 0, maximo = 0, minimo = tempos[0];

    for (int i = 0; i < n; i++) {
        soma += tempos[i];
        if (tempos[i] > maximo) maximo = tempos[i];
        if (tempos[i] < minimo) minimo = tempos[i];
    }

    printf("\n[%s] Media: %ld ms | Max: %ld ms | Min: %ld ms | "
           "Desequilibrio: %ld ms\n",
           modo, soma / n, maximo, minimo, maximo - minimo);
}

/* -----
 * main
 * ----- */
int main(void) {
    pthread_t threads[N_THREADS];

    /* ===== ESTÁTICO ===== */
    printf("=== BALANCEAMENTO ESTATICO ===\n");

    int chunk = N_IMAGENS / N_THREADS;
    for (int i = 0; i < N_THREADS; i++) {
        args_estatico_t *a = malloc(sizeof(args_estatico_t));
        a->id = i;
        a->inicio = i * chunk;
        a->fim = (i == N_THREADS - 1) ? N_IMAGENS : (i + 1) * chunk;
        pthread_create(&threads[i], NULL, worker_estatico, a);
    }
    for (int i = 0; i < N_THREADS; i++) pthread_join(threads[i], NULL);

    analisar("ESTATICO", tempo_estatico, N_THREADS);

    /* ===== DINÂMICO ===== */
    printf("\n=== BALANCEAMENTO DINAMICO (chunk=%d) ===\n", CHUNK_SIZE);
    proximo_dyn = 0; /* Reinicia contador */

    for (int i = 0; i < N_THREADS; i++) {
        args_dinamico_t *a = malloc(sizeof(args_dinamico_t));
        a->id = i;
        pthread_create(&threads[i], NULL, worker_dinamico, a);
    }
    for (int i = 0; i < N_THREADS; i++) pthread_join(threads[i], NULL);

    analisar("DINAMICO", tempo_dinamico, N_THREADS);

    pthread_mutex_destroy(&mutex_dyn);

```

```
    return 0;
}
```

Explicação do Código

1. `custo_ms[]` : Array com custos propositalmente heterogêneos para evidenciar o problema do balanceamento estático com cargas irregulares.
2. `worker_estatico()` : Recebe `inicio` e `fim` fixos; processa seu bloco sem nenhuma sincronização — overhead zero, mas threads rápidas ficam ociosas após terminar.
3. `worker_dinamico()` : Loop infinito com lock mínimo: trava apenas para ler e incrementar `proximo_dyn`, depois processa o chunk inteiramente fora do lock. Isso mantém a seção crítica curtíssima.
4. `CHUNK_SIZE = 2` : Cada thread pega 2 imagens por vez. Valor maior → menos contenção, pior balanceamento. Valor menor → melhor balanceamento, mais contenção.
5. `if (meu_inicio >= N_IMAGENS) break` : Condição de parada verificada *após* liberar o lock, evitando lock desnecessário.
6. `analisar()` : Mede desequilíbrio como `max - min` entre threads; no estático esse valor é alto, no dinâmico tende a ser próximo de zero.
7. `agora_ms()` : Usa `CLOCK_MONOTONIC` para medição precisa do tempo de execução de cada thread.

Saída Esperada

```
=== BALANCEAMENTO ESTATICO ===
[ESTATICO] Thread 0 processou imagem 0 (custo: 300 ms)
[ESTATICO] Thread 1 processou imagem 5 (custo: 15 ms)
...
[ESTATICO] Thread 1 terminou em 335 ms
[ESTATICO] Thread 0 terminou em 1060 ms

[ESTATICO] Media: 710 ms | Max: 1060 ms | Min: 335 ms | Desequilíbrio: 725 ms

=== BALANCEAMENTO DINAMICO (chunk=2) ===
[DINAMICO] Thread 1 processou imagem 2 (custo: 250 ms)
[DINAMICO] Thread 3 processou imagem 0 (custo: 300 ms)
...
[DINAMICO] Thread 2 terminou em 682 ms
[DINAMICO] Thread 0 terminou em 701 ms

[DINAMICO] Media: 690 ms | Max: 701 ms | Min: 672 ms | Desequilíbrio: 29 ms
```

O desequilíbrio cai de ~725 ms para ~29 ms com balanceamento dinâmico — todas as threads terminam aproximadamente no mesmo instante.

Possíveis Problemas

- **Contenção excessiva no mutex dinâmico:** Se `CHUNK_SIZE = 1` e `N` é muito grande, o lock vira gargalo. Solução: aumentar chunk ou usar operações atômicas (`__atomic_fetch_add`).
- **False sharing:** Threads escrevendo em posições de memória próximas (mesmo cache line) causam invalidações de cache constantes. Solução: padding entre arrays de resultado por thread.
- **Overhead de criação de thread maior que o trabalho:** Para tarefas minúsculas, o custo de `pthread_create` supera o ganho. Use thread pools.
- **Imbalance persistente no estático:** Com dados heterogêneos e distribuição estática, a thread mais lenta define o makespan total.
- **Starvation de trabalho (work starvation):** Se uma thread trava em I/O e o trabalho estava estaticamente alocado a ela, outras threads ficam ociosas sem poder ajudar.

Boas Práticas

1. **Meça antes de otimizar:** Use `clock_gettime` para identificar se há desequilíbrio real antes de adicionar complexidade de balanceamento dinâmico.
2. **Ajuste `CHUNK_SIZE` empiricamente:** O valor ótimo depende do hardware, do custo médio das tarefas e do número de threads.

```
/* Heurística de ponto de partida */int chunk_size = (N_ITENS /  
(N_THREADS * 4));if (chunk_size < 1) chunk_size = 1;
```

3. **Minimize o tempo dentro do lock:** Retire tudo o que for possível para fora do mutex; proteja apenas a atualização do contador/fila.
4. **Considere operações atômicas para contadores simples:**

```
#include <stdatomic.h>atomic_int proximo = 0;int meu =  
atomic_fetch_add(&proximo, CHUNK_SIZE);/* Sem mutex! Mais rápido para  
contadores inteiros */
```

5. **Evite false sharing com padding:**

```
typedef struct {    long resultado;    char _pad[64 - sizeof(long)]; /*  
Preenche até 64 bytes (tamanho de cache line) */}  
resultado_t;resultado_t resultados[N_THREADS]; /* Cada thread em sua  
própria cache line */
```

6. **Para trabalho verdadeiramente irregular, use o padrão Saco de Tarefas** (próxima seção).

Comparação com Execução Sequencial

Métrica	Sequencial	Estático (heterogêneo)	Dinâmico
Tempo total	T	T/P + desequilíbrio	$\approx T/P$
Overhead de sincronização	0	0	Baixo ($O(N/\text{chunk})$)
Adaptação a custos variáveis	N/A	Não	Sim
Implementação	Trivial	Simples	Moderada

Resumo Rápido

Balanceamento de Carga

└ Estático: divide N/P itens antes de começar → zero overhead, zero adaptação

└ Dinâmico: threads pegam chunks de fila compartilhada → adaptativo, leve contenção

└ Work-stealing: fila por thread, roubo lazy → mínima contenção, máxima adaptação

└ Chunk size: maior → menos lock, pior balanceamento; menor → melhor balanceamento, mais lock

└ Métrica: desequilíbrio = $\max(\text{tempo_thread}) - \min(\text{tempo_thread})$ → idealmente ≈ 0

Saco de Tarefas (Bag of Tasks)

Definição

Saco de Tarefas (*Bag of Tasks* ou *Task Pool*) é um padrão arquitetural de programação paralela no qual todas as unidades de trabalho são colocadas em uma estrutura centralizada (o "saco"), e um conjunto fixo de threads *worker* retira e executa tarefas dessa estrutura até que ela esteja vazia.

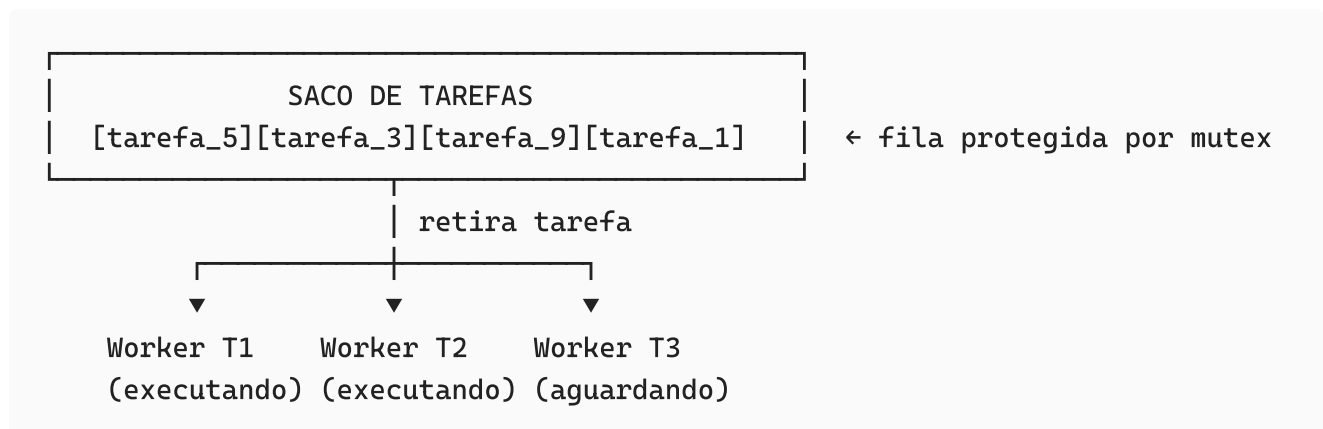
É a realização mais completa do balanceamento dinâmico: o saco funciona ao mesmo tempo como fila de trabalho, mecanismo de balanceamento e ponto de sincronização.

Objetivo

- Desacoplar totalmente a **geração de tarefas** da **execução de tarefas**.
- Adaptar-se automaticamente a tarefas com custos completamente imprevisíveis.
- Reutilizar threads (evitar o custo de criar/destruir threads para cada tarefa).
- Simplificar o código paralelo: thread worker é genérica; a lógica fica nas tarefas.

Funcionamento

Comportamento na memória



Ciclo de vida de uma tarefa

1. **Produção:** Código externo (ou outra tarefa) insere trabalho no saco.
2. **Retirada:** Thread worker acorda (via `pthread_cond_signal`), trava o mutex, retira uma tarefa, libera o mutex.
3. **Execução:** Worker processa a tarefa inteiramente fora do lock.
4. **Retorno (opcional):** Resultado é colocado em estrutura de coleta ou em nova tarefa no saco.
5. **Encerramento:** Quando o saco está vazio e nenhuma nova tarefa será gerada, workers são sinalizados para terminar via flag `encerrado`.

Sincronização

- **Mutex:** Protege a fila de tarefas contra acesso concorrente.
- **Variável de condição:** Workers dormem enquanto o saco está vazio em vez de fazer *busy waiting*.
- **Flag encerrado:** Sinaliza que não virão mais tarefas; workers devem drenar o saco e sair.

Possíveis riscos

- **Deadlock:** Worker retira tarefa → tarefa insere nova tarefa → precisa de lock → lock já está com o worker. Solução: tarefa nunca adquire lock diretamente; usa função `enqueue` que gerencia o lock.
- **Wakeup perdido:** `pthread_cond_signal` disparado antes do worker chamar `wait`. Solução: sempre usar `while` (não `if`) na condição de espera.
- **Encerramento prematuro:** Flag `encerrado` ativada antes de todas as tarefas serem drenadas. Solução: ativar somente após confirmar que nenhuma tarefa pode ser gerada.

Sintaxe

```

/* Estrutura do saco */
typedef struct {
    Tarefa    *fila[MAX_TAREFAS];
    int       cabeca;
    int       cauda;
    int       count;
    int       encerrado;           /* 1 = sem novas tarefas */
    pthread_mutex_t mutex;
    pthread_cond_t  nao_vazio;    /* Workers aguardam aqui */
    pthread_cond_t  nao_cheio;   /* Produtores aguardam aqui */
} Saco;

/* API pública */
void  saco_init   (Saco *s, int capacidade);
void  saco_enqueue(Saco *s, Tarefa *t);    /* Insere tarefa */
int   saco_dequeue(Saco *s, Tarefa **t);   /* Retira tarefa; retorna 0 se
encerrado */
void  saco_fechar (Saco *s);               /* Sinaliza fim da produção */
void  saco_destroy (Saco *s);

```

Parâmetros

Parâmetro / Campo	Tipo	Significado
fila[]	Array de ponteiros	Armazena as tarefas pendentes
cabeca / cauda	int	Índices da fila circular
count	int	Número de tarefas atualmente no saco
encerrado	int (flag)	Quando 1, workers esgotam o saco e saem
nao_vazio	pthread_cond_t	Workers aguardam aqui quando count == 0
nao_cheio	pthread_cond_t	Produtores aguardam aqui quando saco está cheio

Valor de Retorno

```

saco_dequeue(s, &t);
/* Retorna: 1 → tarefa retirada com sucesso
           0 → saco vazio E encerrado → worker deve sair */

```

Exemplo Completo

```

#include <pthread.h>
#include <stdio.h>

```

```

#include <stdlib.h>
#include <unistd.h>

/* -----
 * Padrão Saco de Tarefas (Bag of Tasks)
 * Cenário: processar arquivos com número e custo desconhecidos
 * ----- */

#define MAX_TAREFAS 50
#define N_WORKERS 3

/* --- Definição de uma tarefa --- */
typedef struct {
    int id;
    int custo_ms; /* Simula custo de processamento */
    char descricao[64];
} Tarefa;

/* --- O saco de tarefas --- */
typedef struct {
    Tarefa *fila[MAX_TAREFAS];
    int cabeca;
    int cauda;
    int count;
    int encerrado;
    pthread_mutex_t mutex;
    pthread_cond_t nao_vazio;
    pthread_cond_t nao_cheio;
} Saco;

/* -----
 * Inicialização e destruição do saco
 * ----- */

void saco_init(Saco *s) {
    s->cabeca = 0;
    s->cauda = 0;
    s->count = 0;
    s->encerrado = 0;
    pthread_mutex_init(&s->mutex, NULL);
    pthread_cond_init(&s->nao_vazio, NULL);
    pthread_cond_init(&s->nao_cheio, NULL);
}

void saco_destroy(Saco *s) {
    pthread_mutex_destroy(&s->mutex);
    pthread_cond_destroy(&s->nao_vazio);
    pthread_cond_destroy(&s->nao_cheio);
}

/* -----

```

```

* Insere tarefa no saco (bloqueia se cheio)
* ----- */
void saco_enqueue(Saco *s, Tarefa *t) {
    pthread_mutex_lock(&s->mutex);

    /* Aguarda espaço disponível */
    while (s->count == MAX_TAREFAS && !s->encerrado) {
        printf("[SACO] Cheio, produtor aguardando...\n");
        pthread_cond_wait(&s->nao_cheio, &s->mutex);
    }

    if (!s->encerrado) {
        s->fila[s->cauda] = t;
        s->cauda = (s->cauda + 1) % MAX_TAREFAS;
        s->count++;
        printf("[SACO] Inseriu tarefa %d ('%s') | pendentes: %d\n",
            t->id, t->descricao, s->count);

        pthread_cond_signal(&s->nao_vazio); /* Acorda um worker */
    }

    pthread_mutex_unlock(&s->mutex);
}

/* -----
* Retira tarefa do saco
* Retorna: 1 → tarefa retirada; 0 → encerrado e vazio (worker sai)
* ----- */
int saco_dequeue(Saco *s, Tarefa **t) {
    pthread_mutex_lock(&s->mutex);

    /* Aguarda tarefa disponível OU encerramento */
    while (s->count == 0 && !s->encerrado) {
        pthread_cond_wait(&s->nao_vazio, &s->mutex);
    }

    if (s->count == 0 && s->encerrado) {
        /* Saco vazio e encerrado: worker deve terminar */
        pthread_mutex_unlock(&s->mutex);
        return 0;
    }

    /* Retira da fila circular */
    *t = s->fila[s->cabeca];
    s->cabeca = (s->cabeca + 1) % MAX_TAREFAS;
    s->count--;

    pthread_cond_signal(&s->nao_cheio); /* Acorda produtor se estava
esperando */
    pthread_mutex_unlock(&s->mutex);
}

```

```

        return 1;
    }

/* -----
 * Sinaliza que não virão mais tarefas
 * ----- */
void saco_fechar(Saco *s) {
    pthread_mutex_lock(&s->mutex);
    s->encerrado = 1;
    pthread_cond_broadcast(&s->nao_vazio); /* Acorda todos os workers */
    pthread_mutex_unlock(&s->mutex);
    printf("[SACO] Fechado. Workers drenarao o restante.\n");
}

/* -----
 * Função worker: retira e processa tarefas até o saco fechar
 * ----- */
typedef struct {
    int    id;
    Saco *saco;
    int    n_processadas; /* Estatística local */
} Worker;

void *worker_func(void *arg) {
    Worker *w = (Worker *)arg;
    Tarefa *t;

    printf("[WORKER %d] Iniciado.\n", w->id);

    while (saco_dequeue(w->saco, &t)) {
        /* Processamento fora do lock */
        printf("[WORKER %d] Processando tarefa %d ('%s', %d ms)...\n",
            w->id, t->id, t->descricao, t->custo_ms);

        usleep(t->custo_ms * 1000); /* Simula trabalho real */

        printf("[WORKER %d] Tarefa %d concluida.\n", w->id, t->id);
        w->n_processadas++;
        free(t); /* Libera memória da tarefa */
    }

    printf("[WORKER %d] Encerrando. Processou %d tarefas.\n",
        w->id, w->n_processadas);
    return NULL;
}

/* -----
 * main: produz tarefas e aguarda workers
 * ----- */
int main(void) {

```

```

Saco    saco;
pthread_t workers_tid[N_WORKERS];
Worker  workers[N_WORKERS];

saco_init(&saco);

/* Cria workers ANTES de produzir tarefas */
for (int i = 0; i < N_WORKERS; i++) {
    workers[i].id      = i + 1;
    workers[i].saco    = &saco;
    workers[i].n_processadas = 0;
    pthread_create(&workers_tid[i], NULL, worker_func, &workers[i]);
}

/* Produz tarefas com custos heterogêneos */
int custos[] = {200, 50, 400, 30, 350, 10, 150, 500,
                80, 250, 20, 300, 60, 420, 90};
int n_tarefas = sizeof(custos) / sizeof(custos[0]);

printf("\n[MAIN] Produzindo %d tarefas...\n\n", n_tarefas);

for (int i = 0; i < n_tarefas; i++) {
    Tarefa *t = malloc(sizeof(Tarefa));
    t->id      = i + 1;
    t->custo_ms = custos[i];
    snprintf(t->descricao, sizeof(t->descricao),
             "arquivo_%02d.dat", i + 1);

    saco_enqueue(&saco, t);
    usleep(20000); /* Simula produção não-instantânea */
}

/* Sinaliza fim da produção */
printf("\n[MAIN] Todas as tarefas inseridas.\n");
saco_fechar(&saco);

/* Aguarda todos os workers terminarem */
for (int i = 0; i < N_WORKERS; i++) {
    pthread_join(workers_tid[i], NULL);
}

/* Relatório final */
printf("\n=== RELATORIO FINAL ===\n");
int total = 0;
for (int i = 0; i < N_WORKERS; i++) {
    printf("Worker %d: %d tarefas\n", workers[i].id,
workers[i].n_processadas);
    total += workers[i].n_processadas;
}
printf("Total processado: %d / %d tarefas\n", total, n_tarefas);

```

```
saco_destroy(&saco);  
return 0;  
}
```

Explicação do Código

1. **saco (struct):** Encapsula a fila circular, contador, flag de encerramento e os dois primitivos de sincronização. Toda a concorrência está contida nessa estrutura.
2. **saco_enqueue():** Bloqueia no mutex, aguarda espaço com `pthread_cond_wait(&nao_cheio)` se o saco estiver cheio, insere na posição `cauda` e dispara `signal(&nao_vazio)` para acordar exatamente um worker ocioso. O lock é liberado antes de retornar.
3. **saco_dequeue():** Loop `while (count == 0 && !encerrado)` protege contra *spurious wakeups*. Após a condição ser satisfeita, verifica se deve sair (`count == 0 && encerrado`) ou processar. Retorna `0` para sinalizar ao caller que deve terminar.
4. **saco_fechar():** Usa `pthread_cond_broadcast (não signal)` para garantir que **todos** os workers acordem e verifiquem a condição de saída — caso contrário, alguns poderiam dormir para sempre.
5. **worker_func():** O loop `while (saco_dequeue(...))` é a essência do padrão: retira e processa até o saco retornar `0`. O processamento (`usleep`) ocorre **fora do lock**, maximizando paralelismo.
6. **free(t):** Cada tarefa foi alocada com `malloc` pelo produtor; o worker libera após processar — responsabilidade bem definida.
7. **Workers criados antes das tarefas:** Garante que workers estejam prontos para receber trabalho imediatamente, sem janela de espera.
8. **n_processadas sem lock:** Campo local ao worker, acessado apenas por ele mesmo — sem necessidade de sincronização.

Saída Esperada

```
[WORKER 1] Iniciado.  
[WORKER 2] Iniciado.  
[WORKER 3] Iniciado.  
  
[MAIN] Produzindo 15 tarefas...  
  
[SACO] Inseriu tarefa 1 ('arquivo_01.dat') | pendentes: 1  
[WORKER 1] Processando tarefa 1 ('arquivo_01.dat', 200 ms)...  
[SACO] Inseriu tarefa 2 ('arquivo_02.dat') | pendentes: 1  
[WORKER 2] Processando tarefa 2 ('arquivo_02.dat', 50 ms)...  
[WORKER 2] Tarefa 2 concluída.  
[SACO] Inseriu tarefa 3 ('arquivo_03.dat') | pendentes: 1  
[WORKER 2] Processando tarefa 3 ('arquivo_03.dat', 400 ms)...  
...
```

```
[SACO] Fechado. Workers drenarao o restante.  
[WORKER 1] Encerrando. Processou 5 tarefas.  
[WORKER 3] Encerrando. Processou 5 tarefas.  
[WORKER 2] Encerrando. Processou 5 tarefas.
```

```
=== RELATORIO FINAL ===
```

```
Worker 1: 5 tarefas
```

```
Worker 2: 5 tarefas
```

```
Worker 3: 5 tarefas
```

```
Total processado: 15 / 15 tarefas
```

A distribuição exata entre workers varia a cada execução conforme o escalonamento do SO, mas o total é sempre 15.

Possíveis Problemas

- **Deadlock por tarefa que insere tarefa:** Se um worker chama `saco_enqueue` dentro de `worker_func` enquanto retém o lock interno, haverá deadlock. Solução: `saco_enqueue` usa seu próprio lock interno — desde que o worker não esteja com o lock ao chamar `enqueue`, não há problema. Com a implementação acima isso é seguro porque o worker nunca retém o lock fora de `saco_dequeue`.
- **Memory leak:** Tarefas não processadas ao encerrar (se `saco_fechar` for chamada com tarefas ainda no saco). Nesta implementação os workers drenam tudo antes de sair, mas em encerramento abrupto é necessário um destrutor que libere tarefas remanescentes.
- **Saco ilimitado:** Sem o controle `nao_cheio`, um produtor muito rápido esgota a memória. Sempre defina uma capacidade máxima.
- **Encerramento prematuro:** Chamar `saco_fechar` antes de inserir todas as tarefas; workers verão saco vazio e encerrado e sairão antes de receber todo o trabalho.
- **Número de workers inadequado:** Poucos workers → gargalo; muitos workers → contenção no mutex e overhead de context switch superam o ganho paralelo.

Boas Práticas

1. **Encapsule o saco em uma struct com API bem definida** — `init`, `enqueue`, `dequeue`, `fechar`, `destroy`. Facilita testes e reutilização.
2. **Sempre use `while` na condição de espera**, nunca `if`:

```
/* CORRETO */while (s->count == 0 && !s->encerrado)  
    pthread_cond_wait(&s->nao_vazio, &s->mutex);
```

3. **Use broadcast em `saco_fechar`**, não `signal` — garante que todos os workers acordem.
4. **Processe sempre fora do lock:** Retire a tarefa, libere o mutex, execute; nunca processe dentro do lock.

5. **Defina responsabilidade de free** : Quem libera a memória da tarefa? (Neste exemplo: o worker.) Documente explicitamente.

6. **Dimensione o número de workers pelo hardware:**

```
/* Heurística para tarefas CPU-bound */int n_workers =  
sysconf(_SC_NPROCESSORS_ONLN); /* Para I/O-bound, pode usar 2x ou 4x o  
número de cores */
```

7. **Adicione timeout em saco_dequeue para sistemas que precisam de cancelamento:**

```
struct timespec prazo; clock_gettime(CLOCK_REALTIME, &prazo); prazo.tv_sec  
+= 5; /* Timeout de 5 segundos */ pthread_cond_timedwait(&s->nao_vazio,  
&s->mutex, &prazo);
```

Comparação com Execução Sequencial

Aspecto	Sequencial	Thread por tarefa	Saco de Tarefas
Criação de threads	0	1 por tarefa	P (fixo, reutilizadas)
Balanceamento	N/A	Automático	Automático
Overhead de criação	0	Alto (muitas threads)	Mínimo (threads reusadas)
Adaptação a custos variáveis	N/A	Sim	Sim
Controle de recursos	Total	Difícil (threads ilimitadas)	Preciso (P fixo)
Implementação	Trivial	Simples	Moderada

Erros Frequentes em Provas/Exercícios

- **Usar if em vez de while na espera:** Não protege contra spurious wakeup.
- **Chamar pthread_cond_signal com o mutex destrancado:** Comportamento indefinido.
- **Esquecer pthread_cond_broadcast no saco_fechar :** Workers dormem para sempre.
- **Processar a tarefa dentro do lock:** Serializa toda a execução, eliminando o paralelismo.
- **Não inicializar encerrado = 0 :** Flag lixo pode encerrar workers imediatamente.
- **Liberar a tarefa duas vezes:** Worker libera, depois produtor tenta liberar novamente.
- **Criar workers após inserir tarefas:** Race condition onde saco_fechar pode ser chamado antes dos workers iniciarem.

Resumo Rápido

Saco de Tarefas (Bag of Tasks)

- └─ Estrutura: fila + mutex + cond_ao_vazio + cond_ao_cheio + flag encerrado
- └─ Produtor: saco_enqueue → bloqueia se cheio → signal(ao_vazio)
- └─ Worker: saco_dequeue → bloqueia se vazio → processa FORA do lock
- └─ Encerramento: saco_fechar → broadcast(ao_vazio) → workers drenam e saem
- └─ Vantagem chave: threads reutilizadas + balanceamento automático
- └─ Diferença do Produtor-Consumidor: foco em distribuição de trabalho, não em pipeline de dados; workers são genéricos e o "produto" é a tarefa

Conceito	Sintaxe	Objetivo
pthread_create	pthread_create(&tid, NULL, func, arg)	Criar thread
pthread_join	pthread_join(tid, &retval)	Aguardar término
pthread_exit	pthread_exit(retval)	Terminar thread
Mutex Lock	pthread_mutex_lock(&m)	Entrar seção crítica
Mutex Unlock	pthread_mutex_unlock(&m)	Sair seção crítica
Cond Wait	pthread_cond_wait(&c, &m)	Aguardar condição
Cond Signal	pthread_cond_signal(&c)	Acordar 1 thread
Barrier Wait	pthread_barrier_wait(&b)	Sincronizar fases
Sem Wait	sem_wait(&s)	Decrementar semáforo
Sem Post	sem_post(&s)	Incrementar semáforo

Compilação e Execução

```
gcc -pthread programa.c -o programa
./programa
```

Ou:

```
gcc programa.c -o programa -lpthread
./programa
```

FIM DA DOCUMENTAÇÃO